

Uvod u neuronske mreže

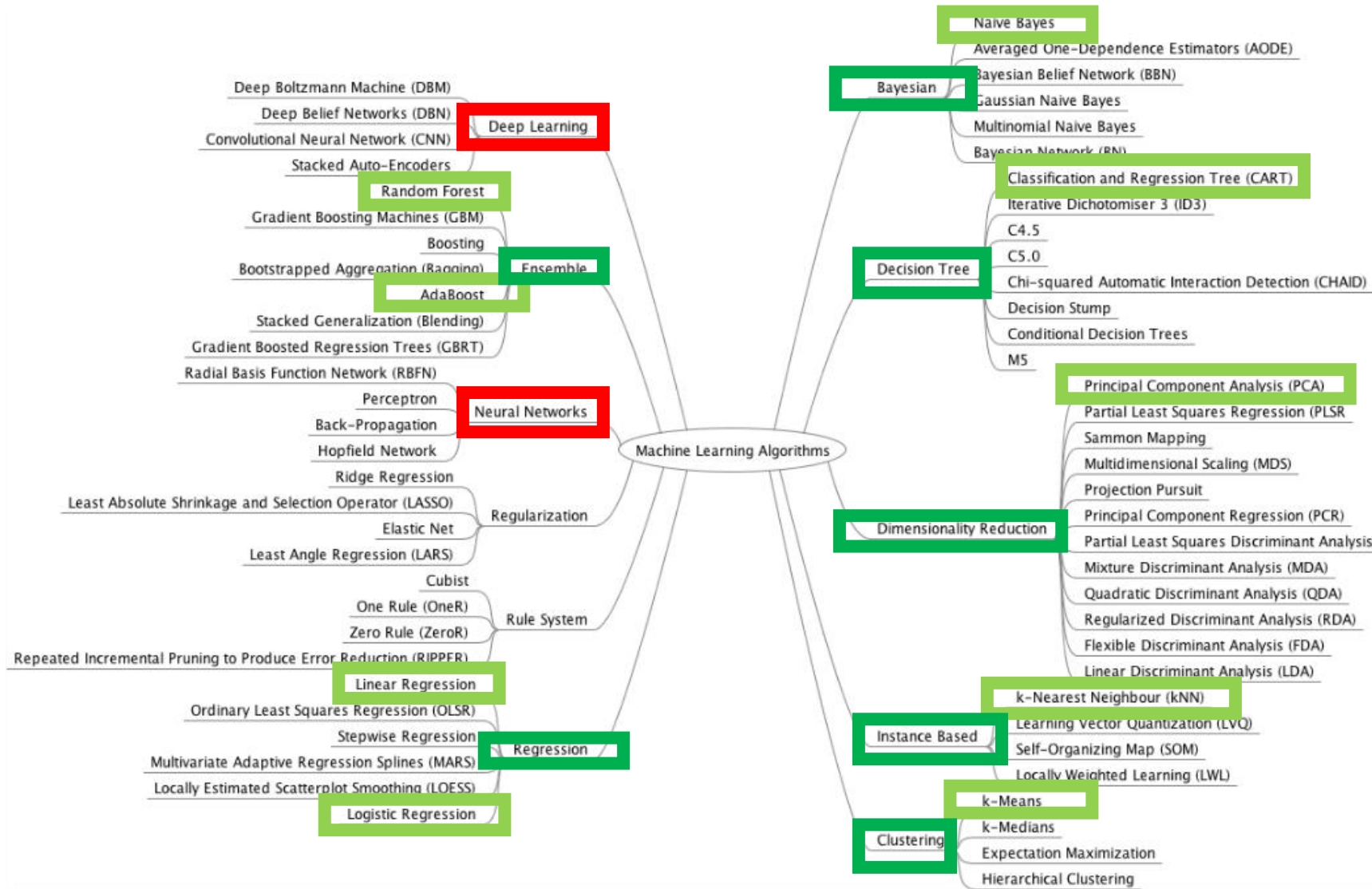


Sadržaj

- Što do sada znamo/ne znamo?
- Motivacija za korištenje neuronskih mreža
- Umjetne neuronske mreže
- Učenje neuronske mreže
 - ***Back-propagation*** algoritam

Što do sada znamo/ne znamo?

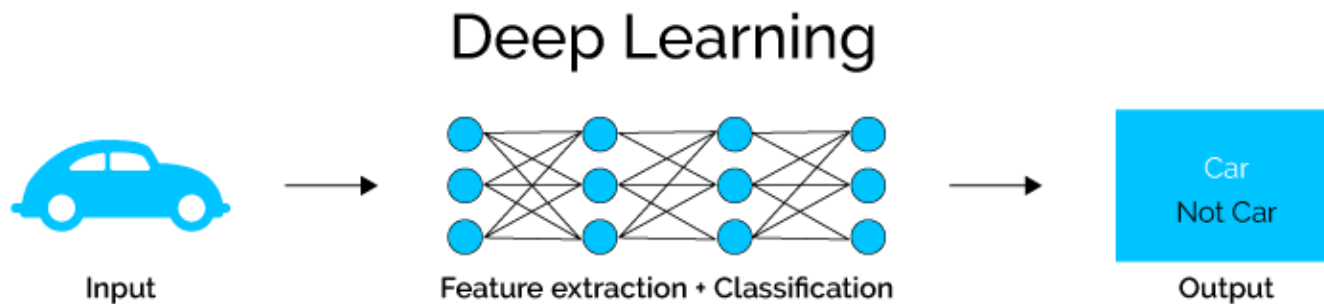
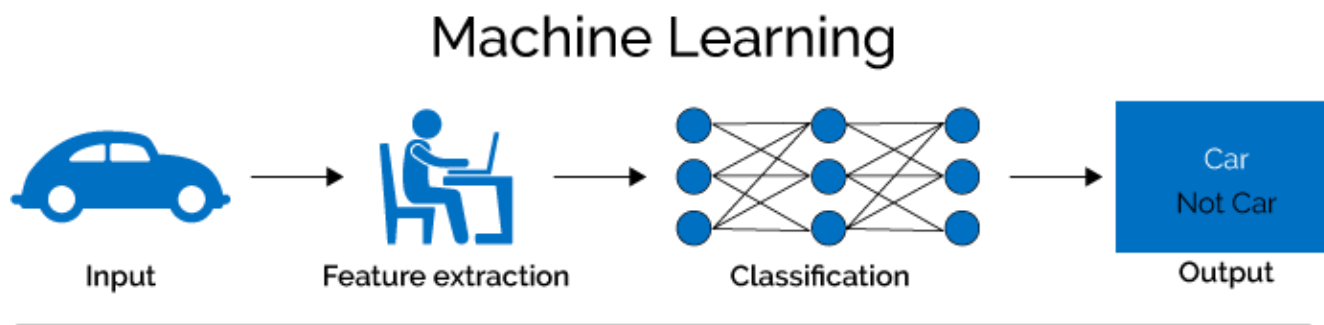
Podsjetnik... Podjela metoda strojnog učenja



[A Tour of Machine Learning Algorithms](#)

Tradicionalni računalni vid i duboko učenje

- „Tradicionalni“ računalni vid zasniva se na ručnom izdvajanju značajki te učenju klasifikatora na temelju podataka
- Moderni pristup zasniva se na „*end-to-end learning*“ – značajke se također „uče“ iz dostupnih podataka



Tradicionalni računalni vid i duboko učenje

- Značajke na temelju kojih se radi klasifikacija slika:
 - *Scale-Invariant Feature Transform (SIFT)*
 - *Speeded-Up Robust Features (SURF)*
 - *Histogram of Oriented Gradients (HOG)*
 - *Local Binary Patterns (LBP)*
 - Histogrami boje
 - ...

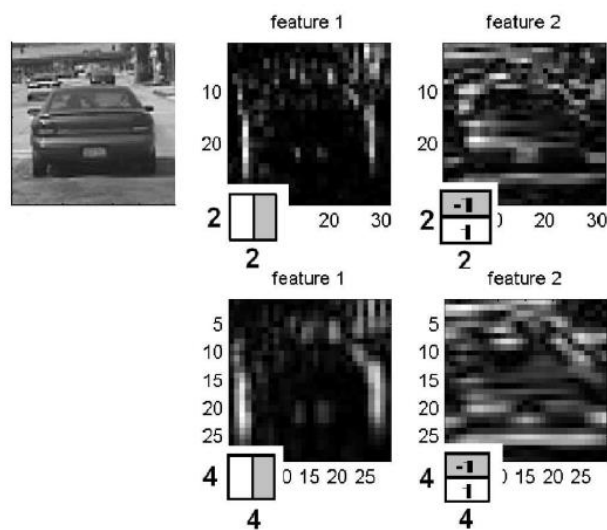


Figure 2: 2D Haar Wavelet example on a vehicle image.

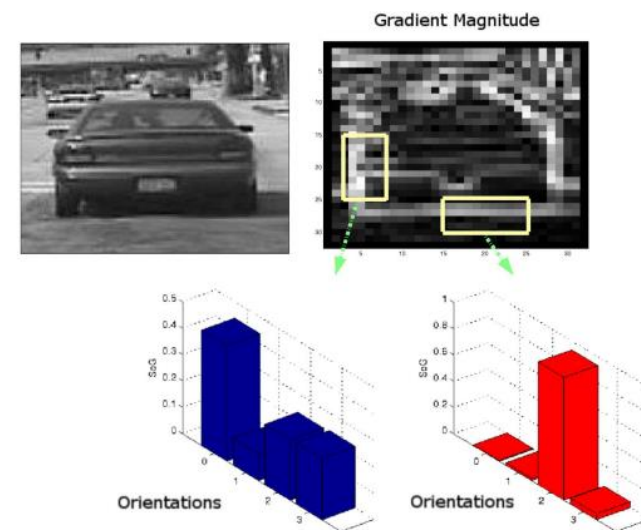


Figure 3: HoG example on a vehicle image.

Tradicionalni računalni vid i duboko učenje

- Složen programski kod;
velik broj parametara
koje je potrebno podesiti
- Vrlo čest pristup:
 - HOG + SVM
 - Viola-Jones algoritam

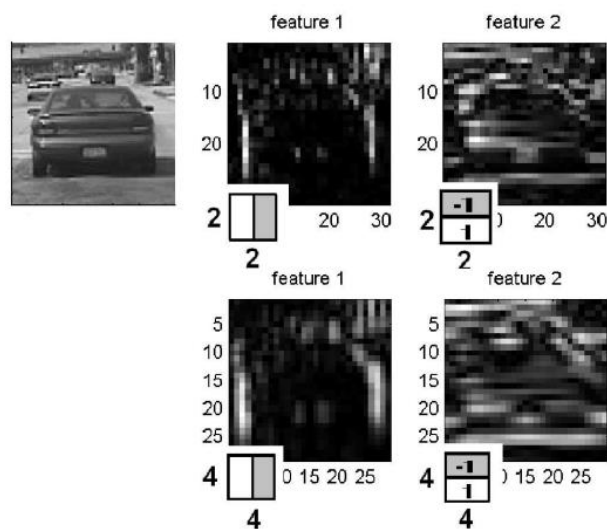


Figure 2: 2D Haar Wavelet example on a vehicle image.

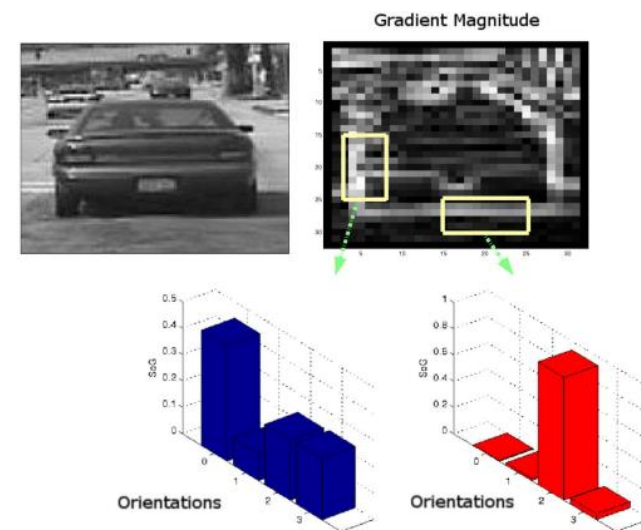
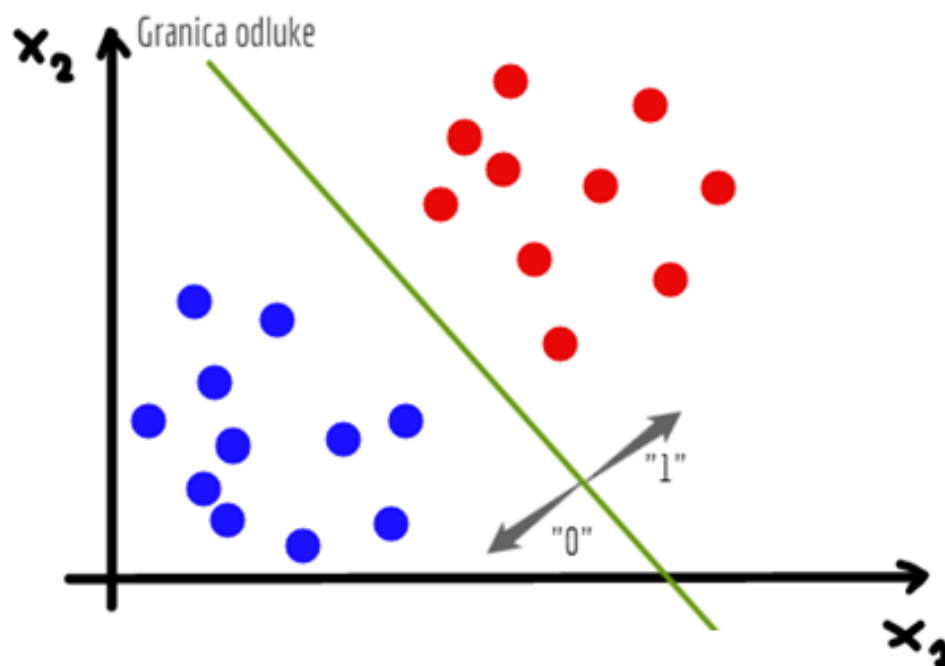


Figure 3: HoG example on a vehicle image.

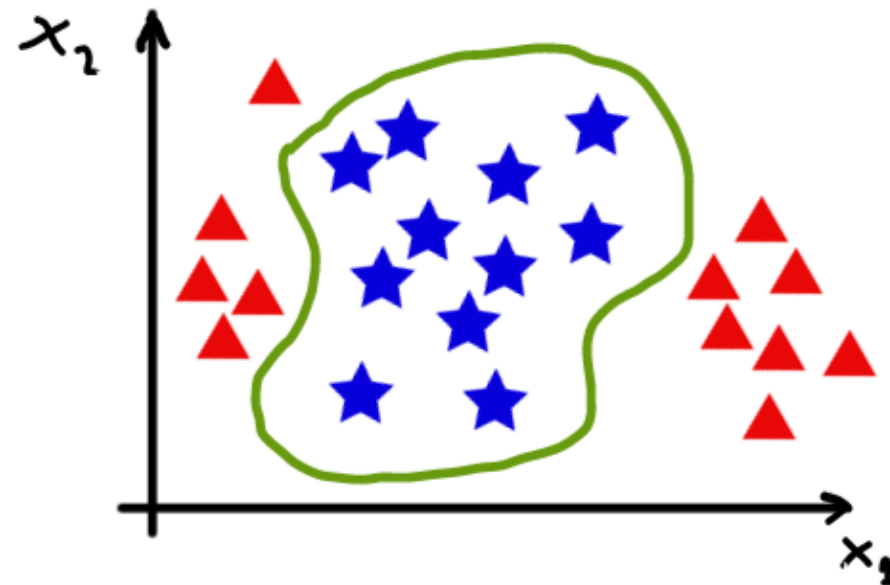
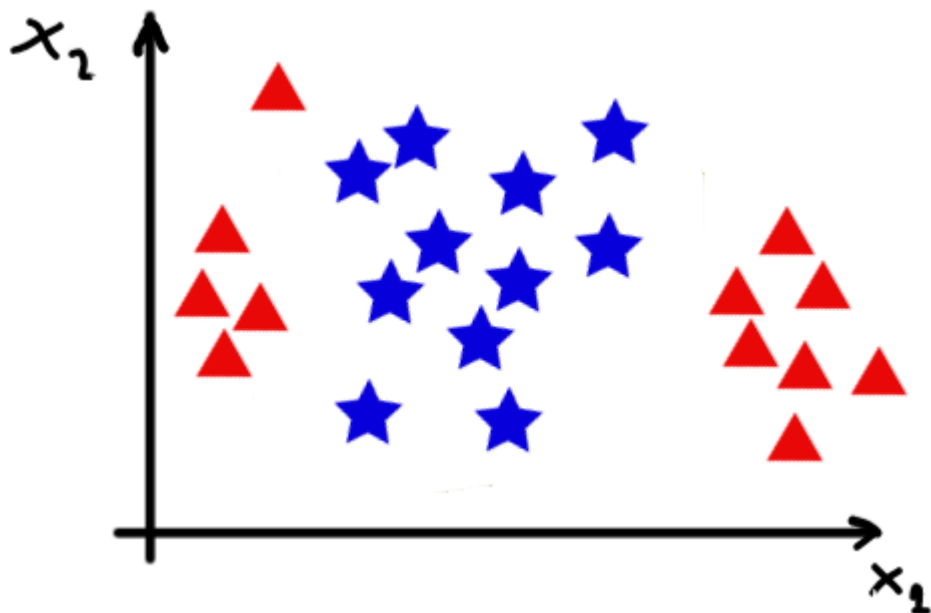
Motivacija za korišćenje neuronskih mreža

Podsjetnik... Logistička regresija

- Do sada smo analizirali modele za regresiju i klasifikaciju koji su bili linearna kombinacija ulaznih veličina
- Npr. klasifikator, logistička regresija



Proširenje logističke regresije

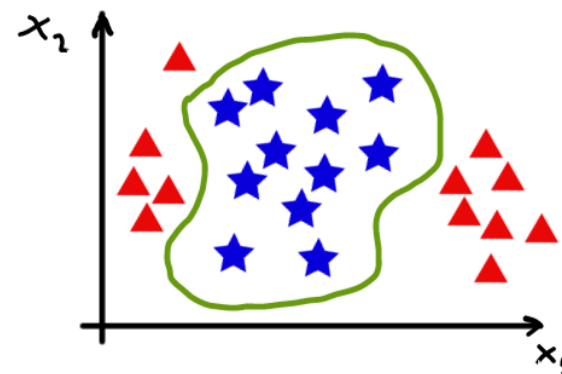


- Može li se takav klasifikator primijeniti na ovaj slučaj?
- Proširenje logističke regresije → matematičkom transformacijom ulaznih veličina moguće je dobiti nelinearnu granicu odluke

$$h_{\theta}(\mathbf{x}^{(i)}) = g(\Theta_0 + \Theta_1 \mathbf{x}_1 + \Theta_2 \mathbf{x}_2 + \Theta_3 \mathbf{x}_1^2 + \Theta_4 \mathbf{x}_2^2 + \Theta_5 \mathbf{x}_1 \mathbf{x}_2 + \Theta_6 \mathbf{x}_1^2 \mathbf{x}_2 + \dots)$$

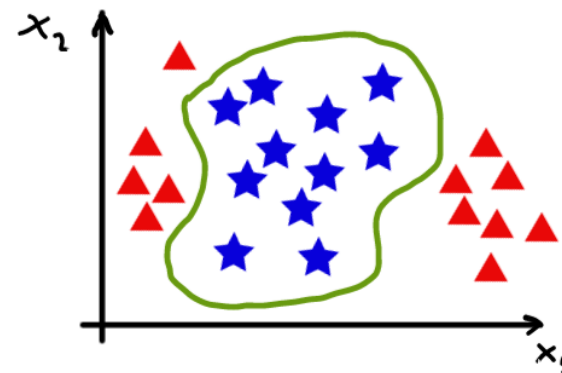
Proširenje logističke regresije

- Ovaj pristup je prihvatljiv ako je mali broj ulaznih veličina x_i (npr. 2-3)
- Za slučaj većeg broja ulaznih veličina, npr.
 - $n = 200 \rightarrow$ ako se žele uključiti svi kvadratni članovi otprilike 200 parametara
 - $n = 100 \rightarrow$ ako se žele uključiti sve kombinacije do kubičnih članova otprilike 170000 parametara
 - Slika 50 x 50 piksela (2500 ulaznih veličina); u slučaju RGB slike je to $3 \times 2500 = 7500$ ulaznih veličina $\rightarrow$ ako se žele uključiti sve kombinacije do kubičnih članova to je nekoliko milijuna parametara
- Zbog velikog broja parametara može doći do *overfittinga*



Proširenje logističke regresije

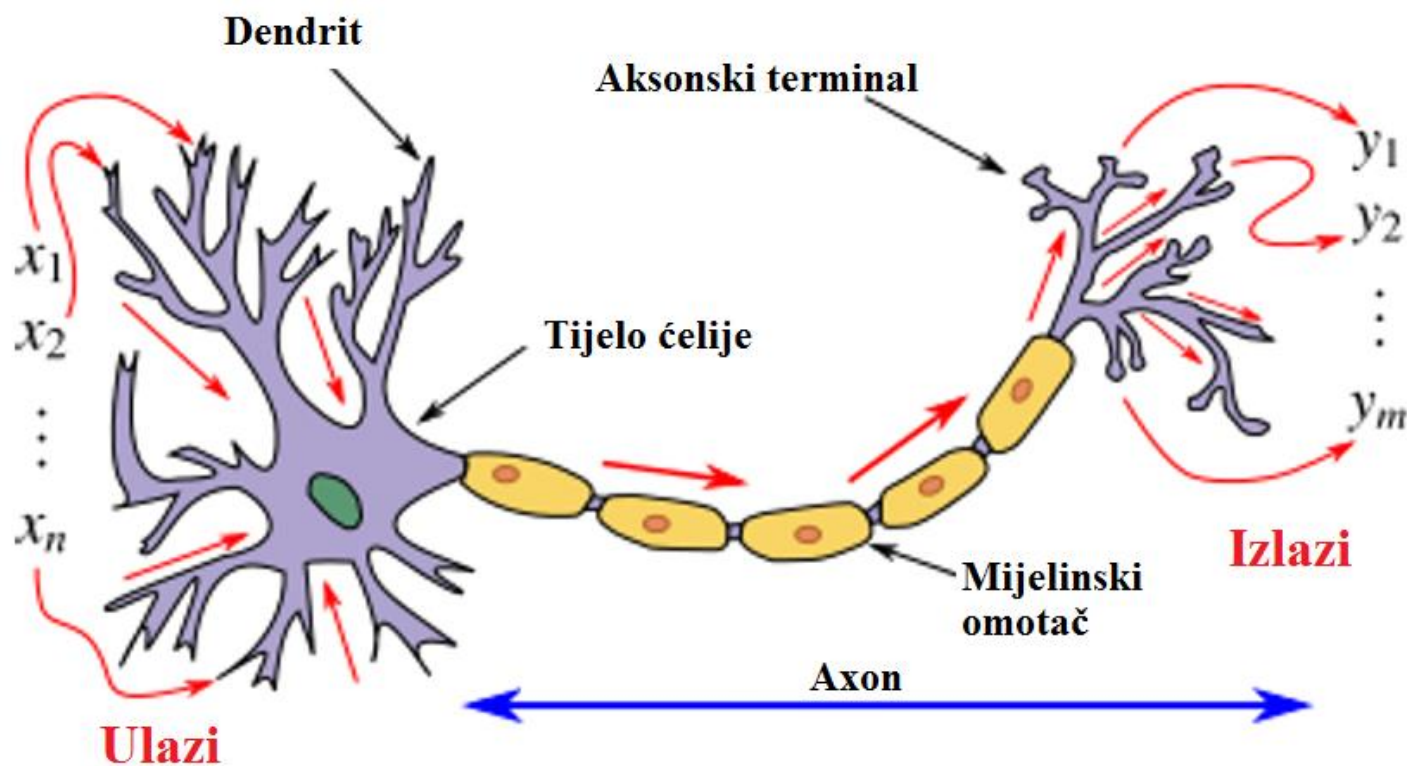
- Moguće je uključiti samo neke transformacije ulazne veličine → postavlja se pitanje koliko složenu granicu odluke možemo dobiti na taj način
- Jedan od načina je fiksiranje broja baznih funkcija unaprijed, ali se pokazalo da to nije uvijek učinkovito
- Zaključak – kada ulazni podatak karakterizira velika količina različitih podataka (kao npr. kod slike), onda metode koje smo do sada predstavili neće biti učinkovite



Umjetne neuronske mreže

Umjetne neuronske mreže...inspiracija

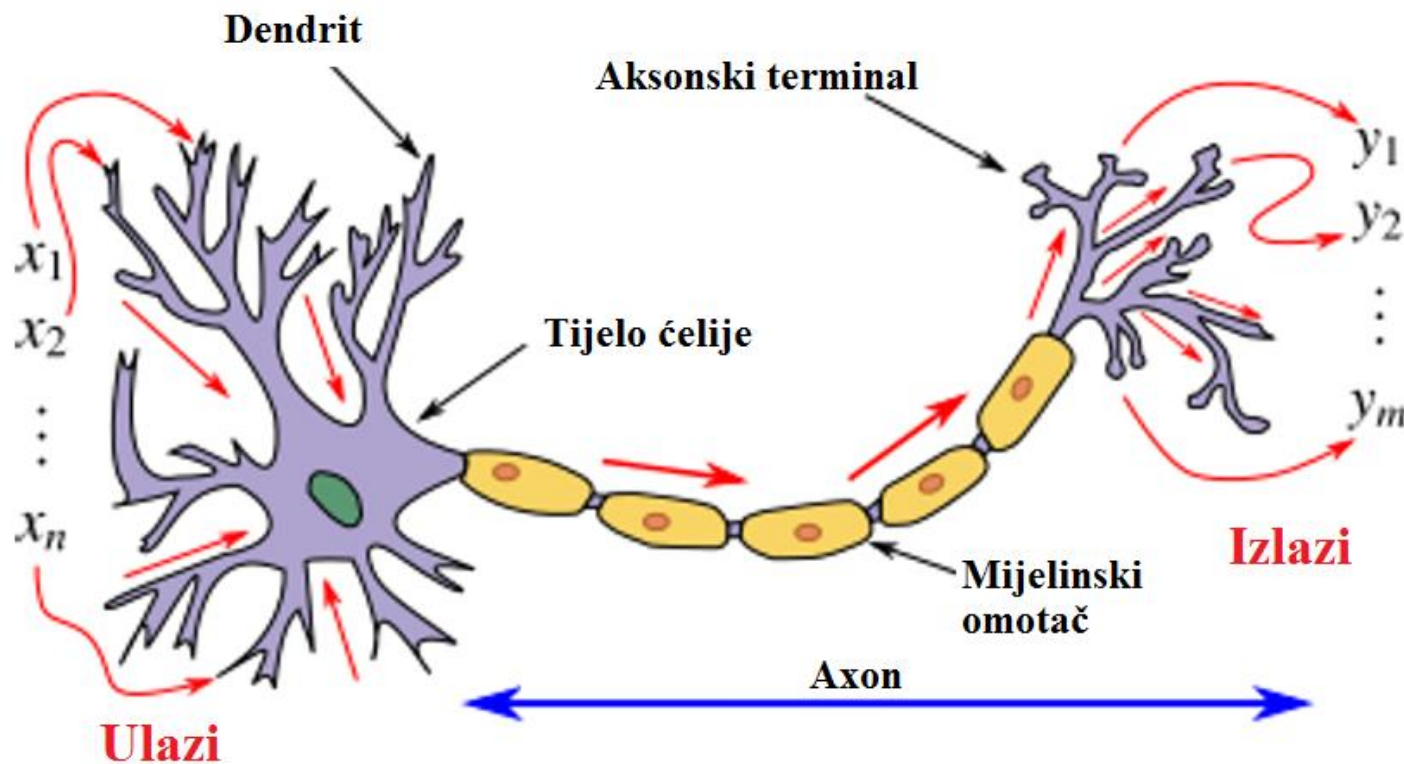
- Promatranje ljudskog mozga inspiriralo je koncept umjetne neuronske mreže (engl. *Artificial Neural Networks* – ANN)



- Neuron – osnovna jedinica procesiranja u mozgu
- Lijevo je jedan neuron (ćelija)
- Tijelo neurona okruženo dendritima
- Neuroni na krajevima svojih dendrita „osluškuju” dolazne signale

Umjetne neuronske mreže...inspiracija

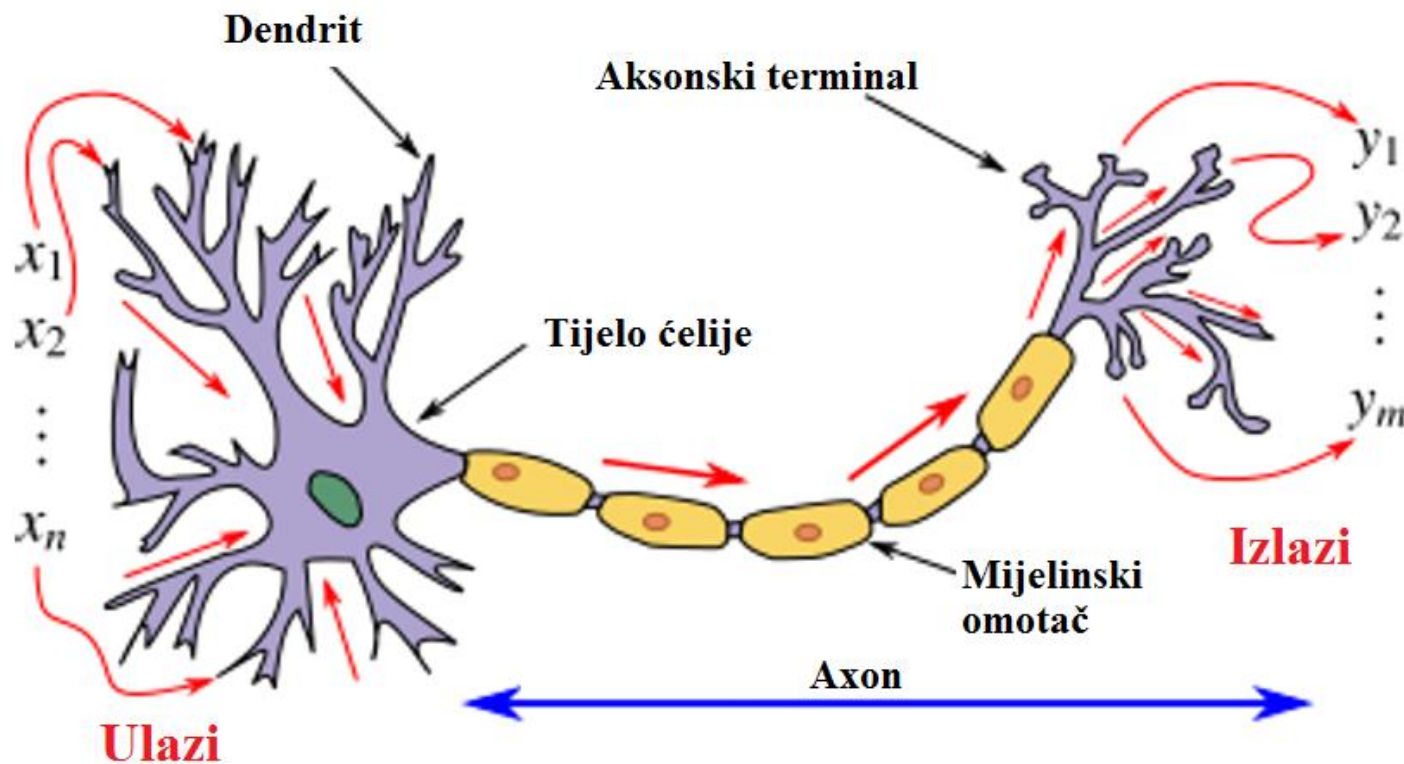
- Promatranje ljudskog mozga inspiriralo je koncept umjetne neuronske mreže (engl. *Artificial Neural Networks* – ANN)



- Neuron je svojim dendritima vezan za izlaze velikog broja (na tisuće) drugih neurona
- Prirodno je da ne utječu svi ti ulazi jednako na reakciju tj. okidanje tog neurona (imaju $\neq$ jačine veze)

Umjetne neuronske mreže...inspiracija

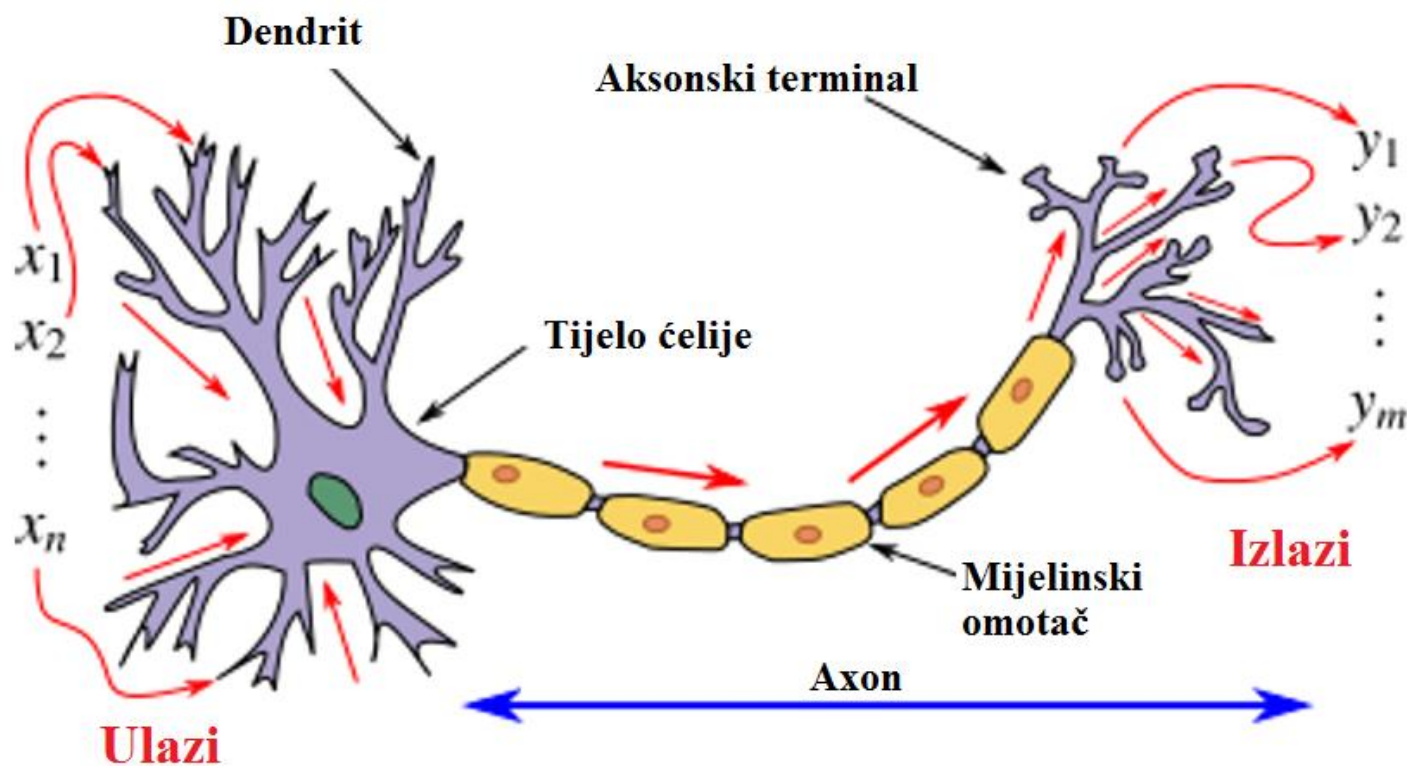
- Promatranje ljudskog mozga inspiriralo je koncept umjetne neuronske mreže (engl. *Artificial Neural Networks* – ANN)



- Neki ulazi su pobuđujući („+“), a neki inhibitorni („-“)
- Svaki neuron ima neku funkciju → suma svih tih ulaza (pobuda) uspoređena s nekim pragom pobudnog signala → neuron „okine“ ili ne

Umjetne neuronske mreže...inspiracija

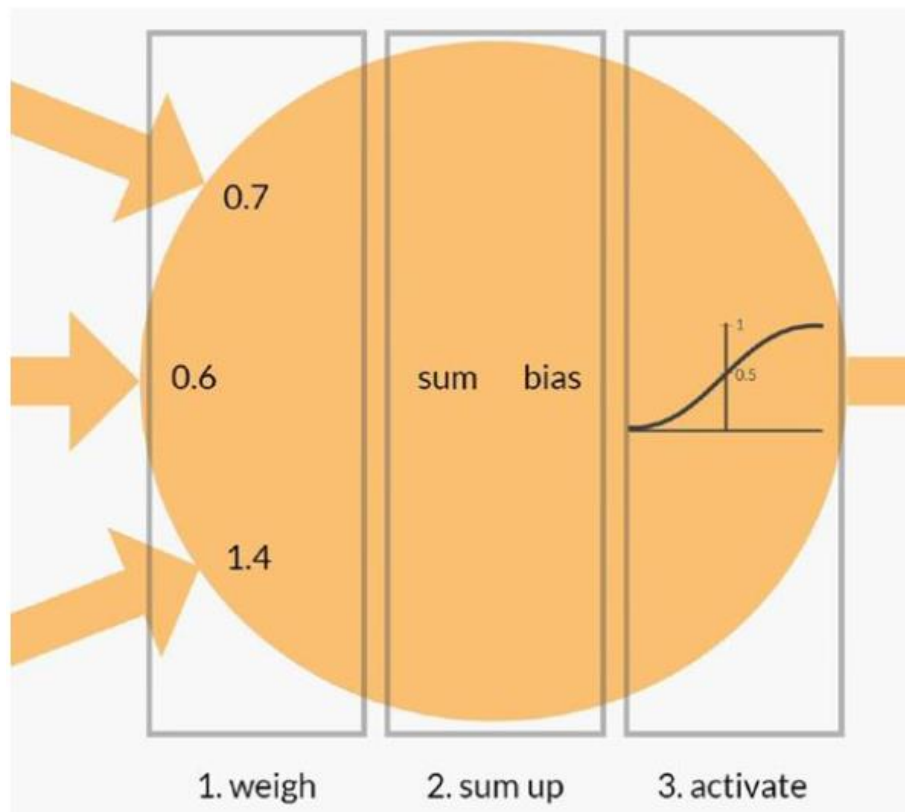
- Promatranje ljudskog mozga inspiriralo je koncept umjetne neuronske mreže (engl. *Artificial Neural Networks* – ANN)



- Ako okine $\rightarrow$ nekakav signal se fizički propagira kroz akson $\rightarrow$ kada dođe do kraja aksona tog neurona, fizički se razgrana i čini ulaze dendrita sljedećih neurona

Umjetne neuronske mreže...PERCEPTRON

- PERCEPTRON → osnovna jedinica umjetnih neuronskih mreža (ANN)

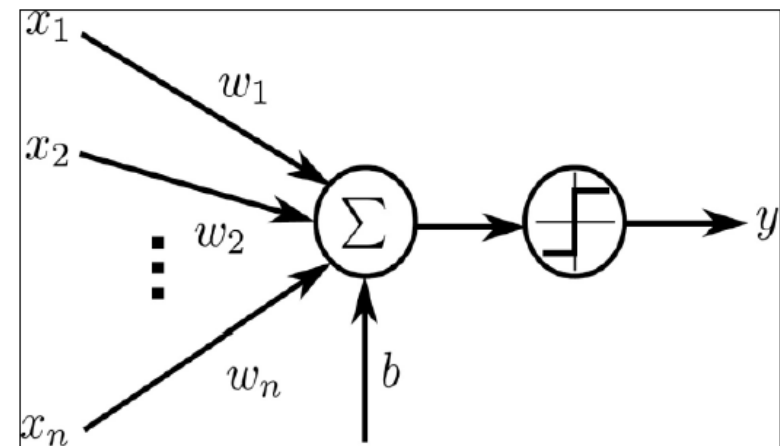
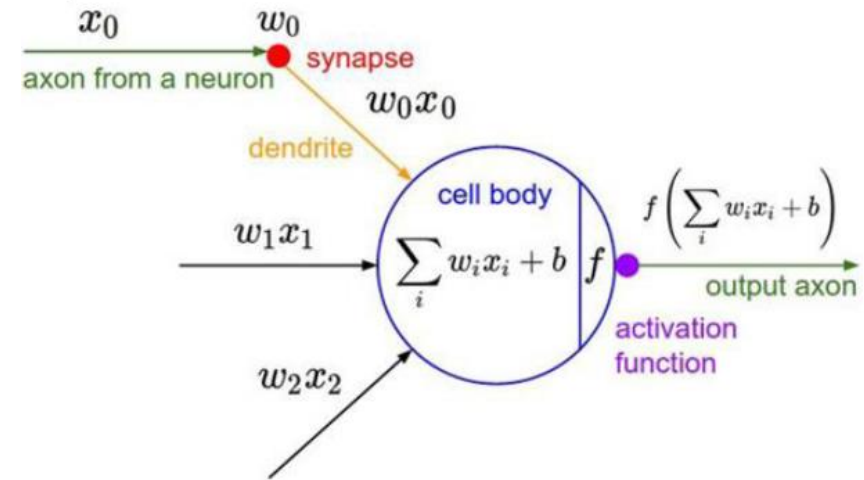
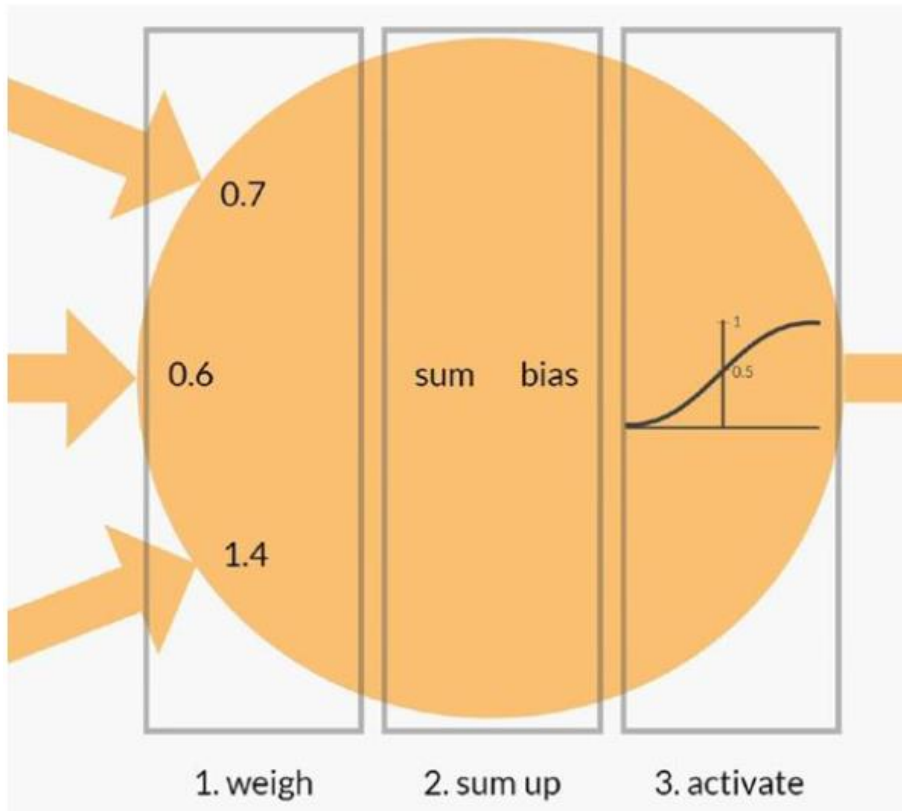


- U suštini odgovara percepciji → kako se nešto procesira i opaža

Ovaj primjer → ulaz je vektor s 3 elementa (inače je ulaz nešto s puno više elemenata!!!)

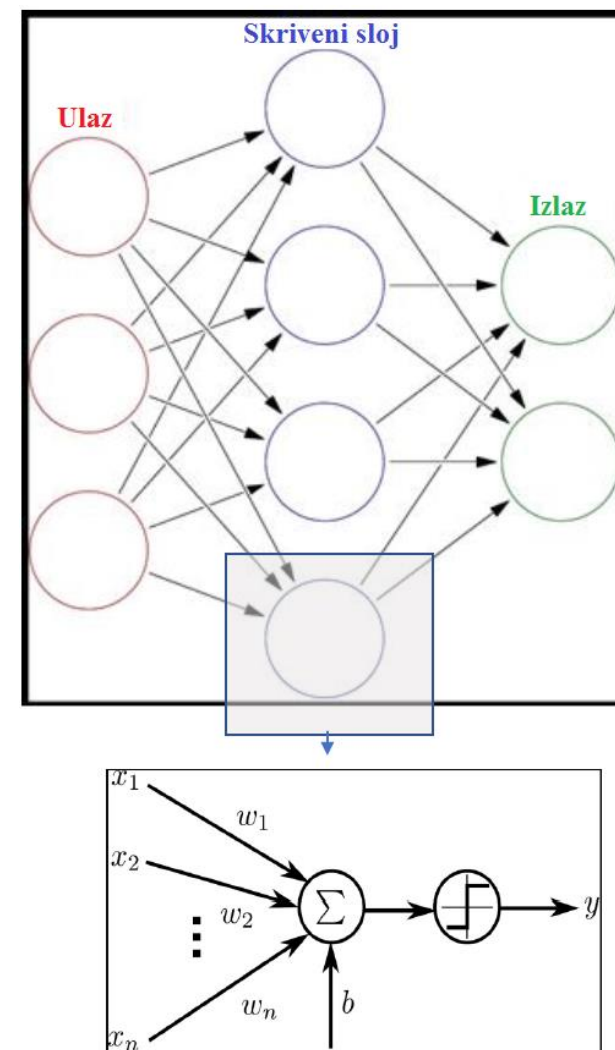
- Svaki se ulaz množi s odgovarajućom težinom
- Sumiranje umnožaka (i opcionalno dodavanje *biasa*)
- Sve dosad nabrojane operacije su linearne
- Na kraju sve skupa prolazi kroz nekakvu aktivacijsku funkciju koja je najčešće nelinearna (mogu biti različite funkcije) → ta funkcija općenito ima formu nekakvog praga (*thresholding*)

Umjetne neuronske mreže...PERCEPTRON



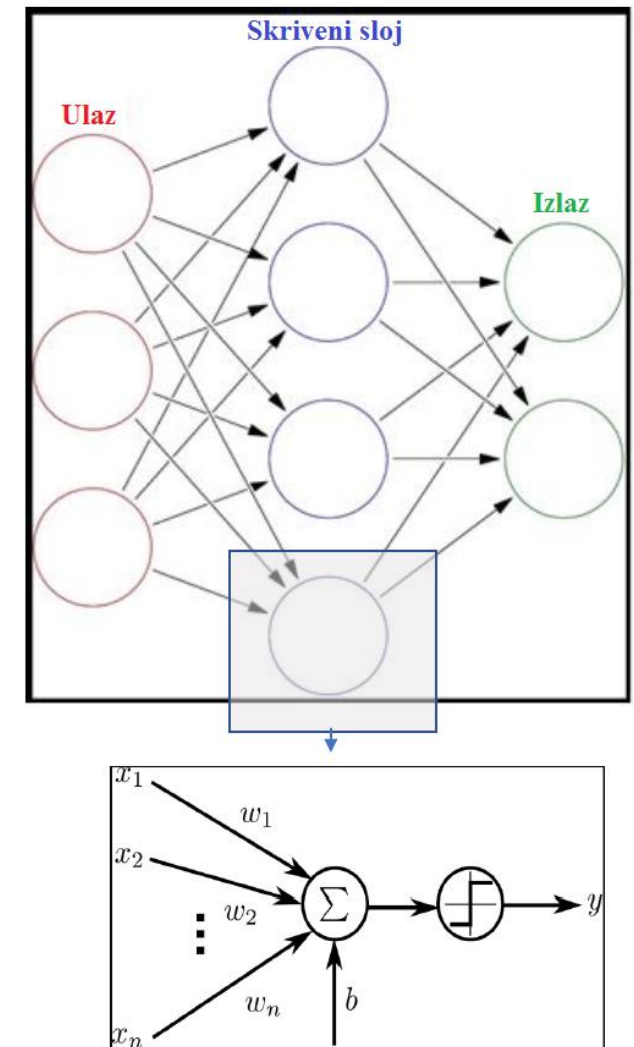
Umjetne neuronske mreže (ANN)

- Umjetna neuronska mreža sastavljena je od niza jednostavnijih (umjetnih čvorova) koji se nazivaju neuroni (procesirajući elementi)
- Sličnost s biološkom neuronskom mrežom:
 - Paralelno procesiranje informacije
 - Mnogo međusobnih veza
- Svaki neuron ima svoju aktivacijsku funkciju
- Ulazni neuroni su aktivirani preko senzora
- Ostali slojevi neurona aktivirani su od prethodno aktiviranih neurona
- Aktivacijska funkcija – aktivira izlaz za neku kombinaciju ulaza



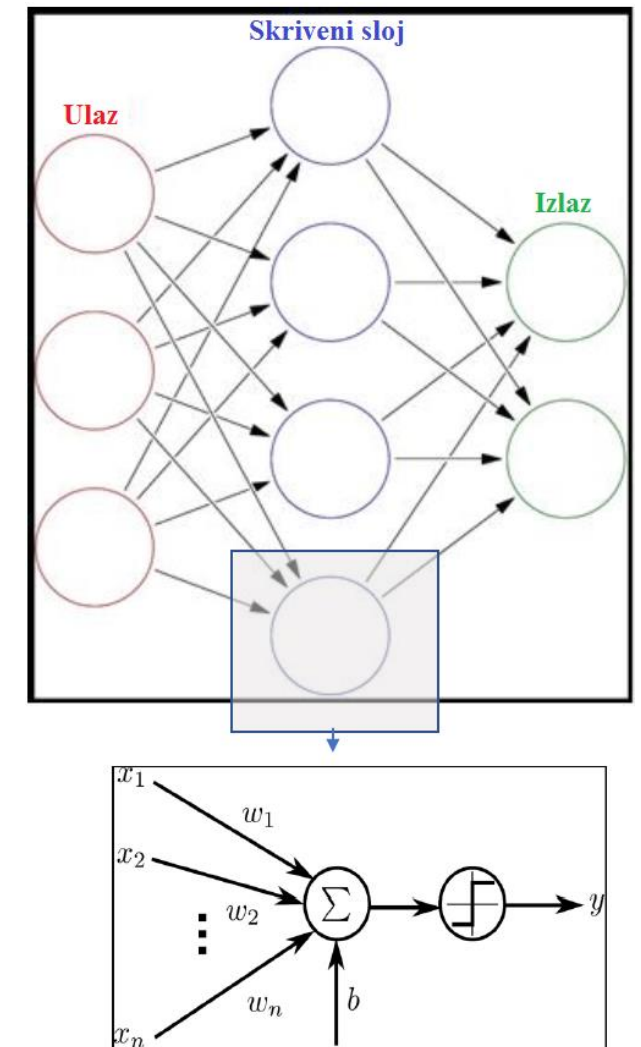
Umjetne neuronske mreže (ANN)

- Ovdje imamo 3 ulaza (to nisu neuroni, nego ulazi), 2 izlazna neurona i 1 skriveni sloj sa 4 neurona
- Ovdje bi ulaz bio vektor $[x_1 \ x_2 \ x_3]$ i svaki ulaz npr. odgovara jednom broju (8-bitnom, 16-bitnom,...)
- Iz svakog ulaza idu „strelice“, tj. veze, u sljedeći sloj u kojem se događaju operacije opisane ranije
- Svaki je ulaz prije „dolaska“ do svakog neurona pomnožen nekim koeficijentom (težinom) w , a može biti dodan i *bias* (detalji na vježbama)
- Rezultat prolazi kroz neku nelinearnu aktivacijsku funkciju (ovdje *sign* f-ja) → rezultat se propagira/ne propagira



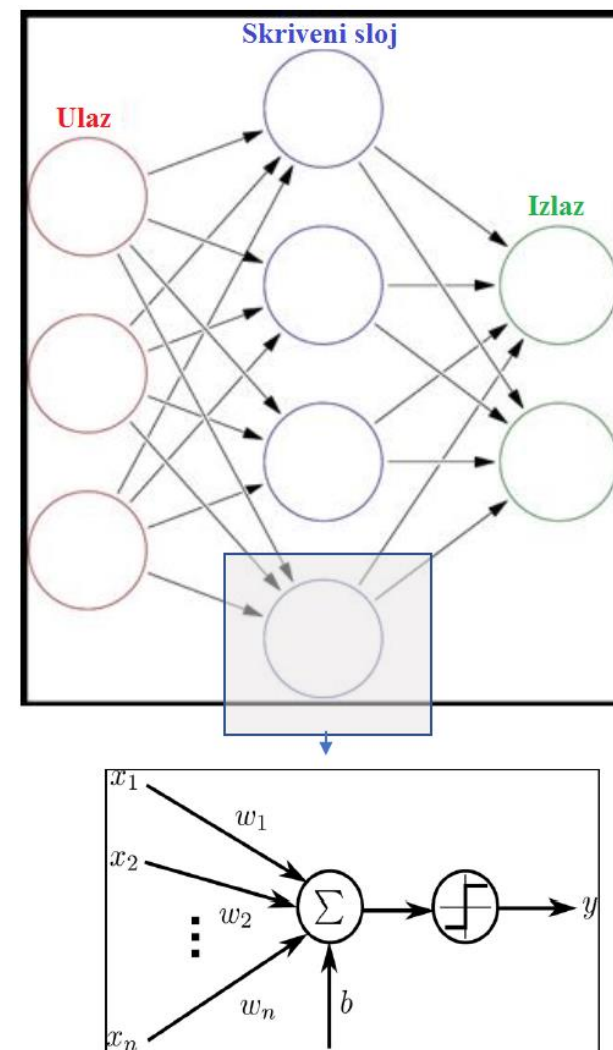
Umjetne neuronske mreže (ANN)

- U svakom sloju imamo niz neurona
- Svaka strelica „odgovara” jednom koeficijentu w_{ij}
- Zasad sve izgleda relativno jednostavno → samo jedan sloj neurona (perceptrona) između ulaza i izlaza
- Općenito → stvar je puno kompliciranija
- Cijela ANN je zapravo funkcija
 - Uzima ulaze i daje nekakav izlaz za te ulaze (multi-dimenzionalna funkcija)
 - Važno da je cijela ta funkcija fundamentalno diferencijabilna (da ima gradijent u odnosu na sve svoje parametre w_{ij})
 - Koliko imamo različitih w_{ij} u ovom primjeru gore desno?



Umjetne neuronske mreže (ANN)

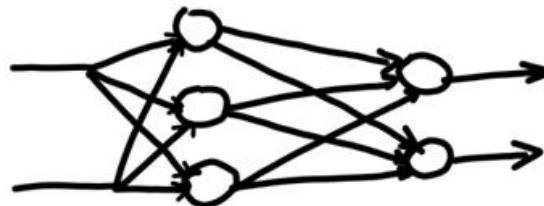
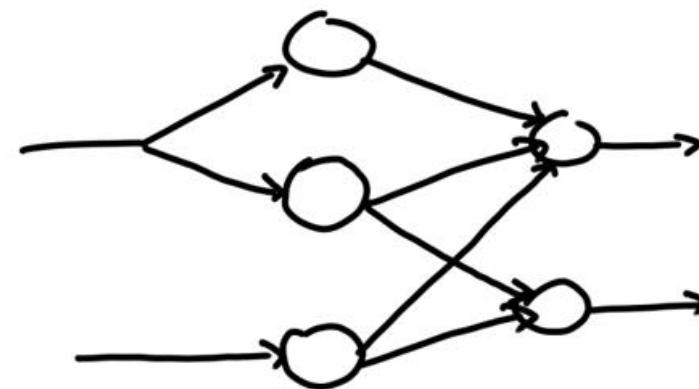
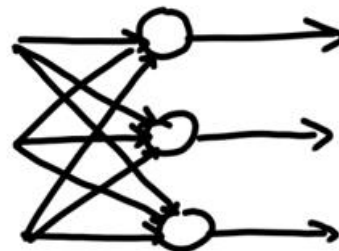
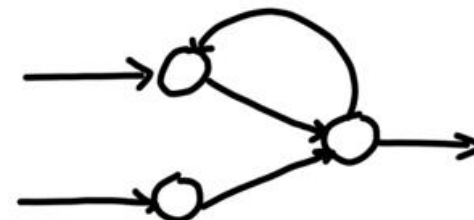
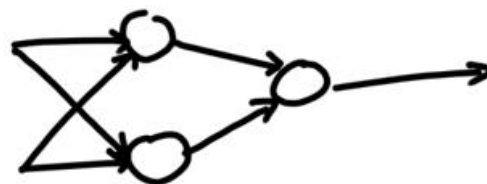
- Vrlo slično ovome radi naš mozak, samo s više ovih skrivenih slojeva
- ANN obavlja neku funkciju koju ja želim (prevođenje s FRA na HR, klasifikacija objekata u prometu, određivanje brzine vozila na temelju nekih ulaznih parametara,...)
- Vidjet ćemo kako „natjeramo” NN da radi baš to što mi hoćemo
- Pričali smo o gradijentu i gradijentnoj metodi ranije na predavanju → koristit ćemo ju i kod ANN
- U ovom primjeru imamo 20 koeficijenata (w_{ij}) koje treba „trenirati”, a mreža je doista jednostavna
 - Koliko smo parametara imali kod linearne regresije za trenirati?



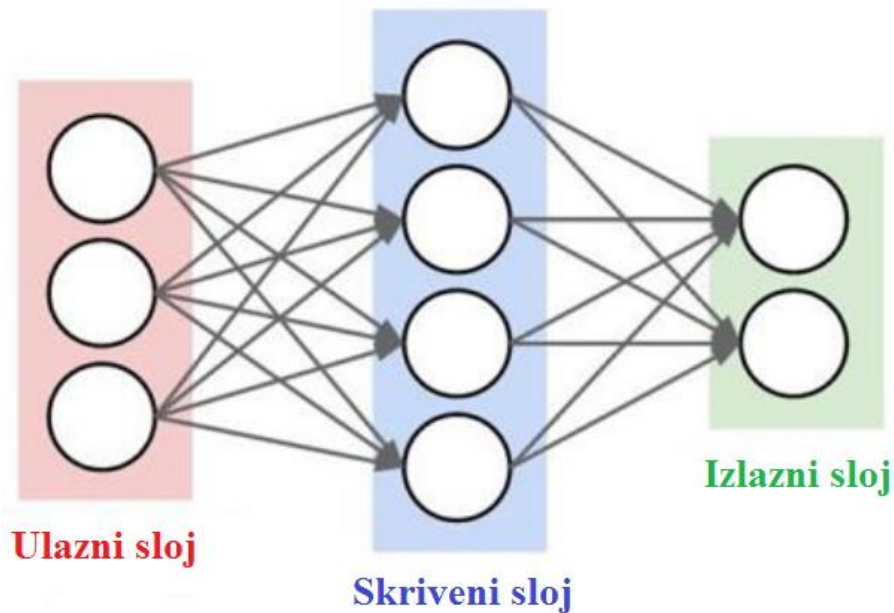
Tipovi neuronskih mreža

Prema načinu spajanja

- Unaprijedne (engl. *Feedforward NN*)
- Cikličke (sadrže povratne veze)
- Jednoslojne ili višeslojne
- Djelomično ili potpuno povezane



Višeslojne NN, Duboke NN (DNN)

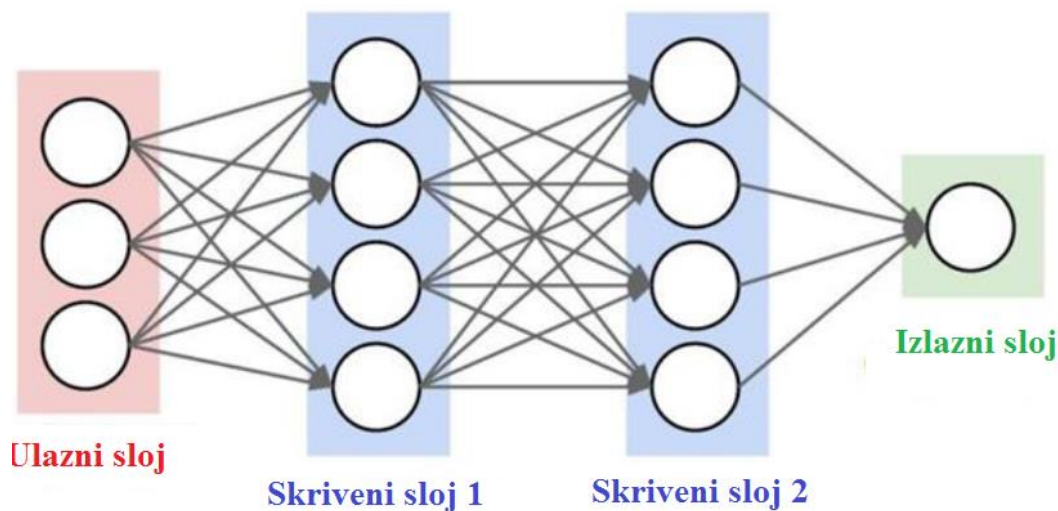


Neuronska mreža s 2 sloja

Mreža s 1 skrivenim slojem

- Skriveni sloj → nije izravno izložen ulazu
- Ovo je plitka mreža – ima samo jedan skriveni sloj
- Osnovna komplikacija → dodavati sve više skrivenih slojeva (ali to je ujedno i više procesiranja)

Višeslojne NN, Duboke NN (DNN)

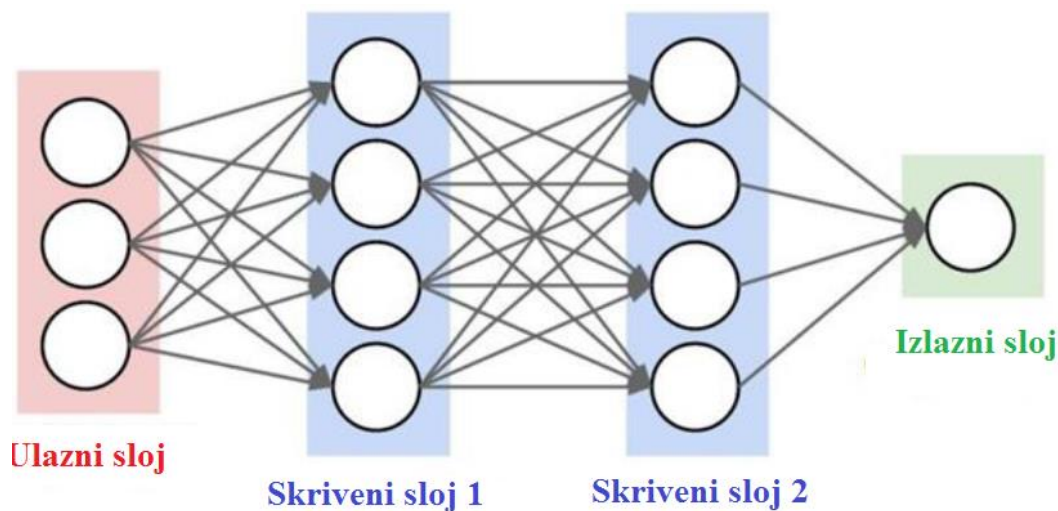


Neuronska mreža s 3 sloja

Mreža s 2 skrivena sloja

- Dodali smo 1 skriveni sloj → duboka mreža
- Zašto dodavati dodatne slojeve?
 - Da bi oponašali sve složenije funkcije koje trebaju točno i precizno raditi?
- Raste broj koeficijenata w_{ij} → gradijentnom metodom naći najbolje koeficijente za dani problem

Višeslojne NN, Duboke NN (DNN)

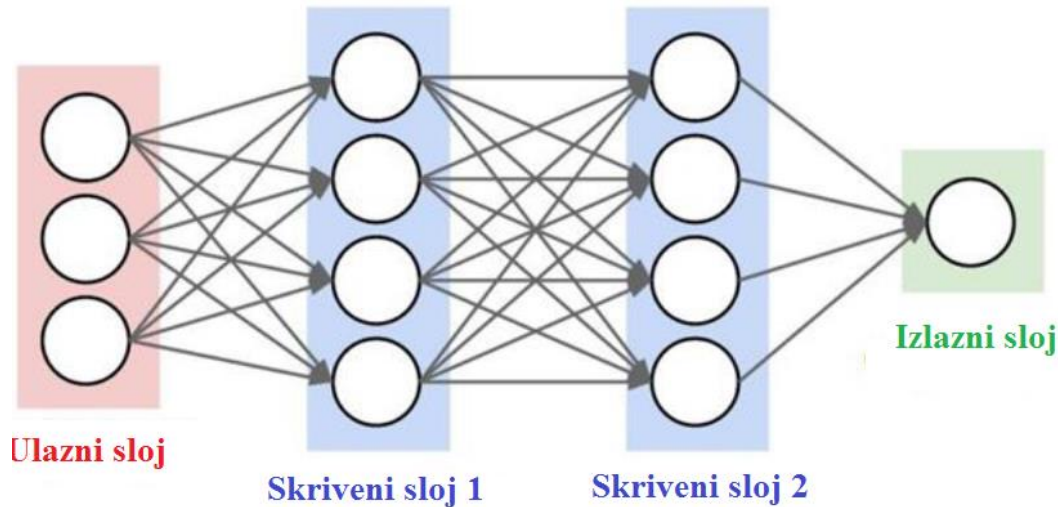


Neuronska mreža s 3 sloja

Mreža s 2 skrivena sloja

- I ovdje se obično kreće sa sasvim slučajnim odabirom početnih parametara w_{ij}
- Dodavanjem broja slojeva i funkcija je sve moćnija i moćnija
- Kako još možemo povećati kompleksnost, a time i sposobnost ANN, osim dodavanja slojeva?
 - Povećavanjem broja neurona u sloju
- Broj slojeva i neurona po sloju → određuju kapacitet mreže (modela)
- DNN → imaju više od jednog skrivenog sloja

Unaprijedne NN



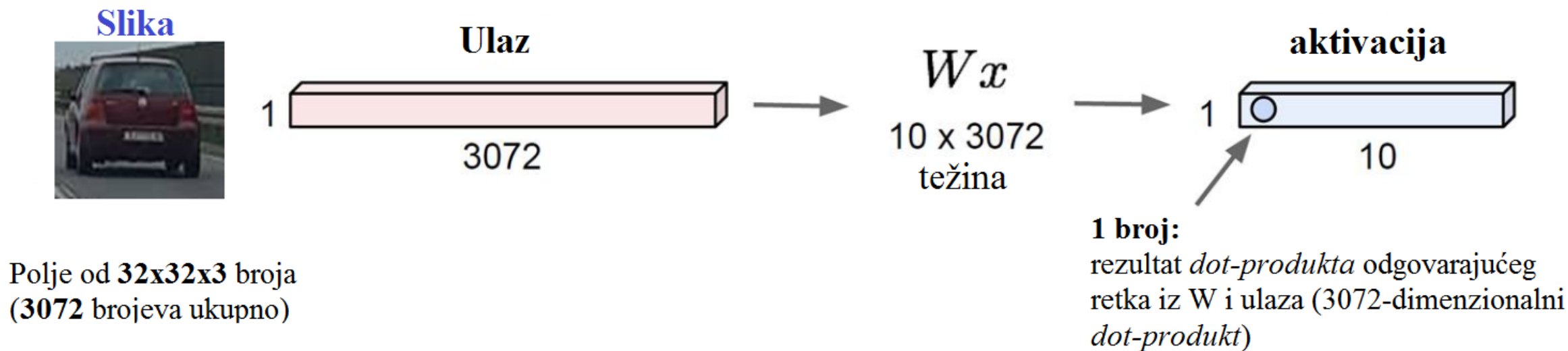
Neuronska mreža s 3 sloja

Mreža s 2 skrivena sloja

- Najčešće se koriste višeslojne unaprijedne neuronske mreže
- Unaprijedne neuronske mreže su mreže u kojima se propagacija signala odvija samo u jednom smjeru, od ulaznih veličina prema izlaznoj veličina, što znači da mreža ne sadrži povratne veze

Potpuno povezani sloj (*fully-connected layer*)

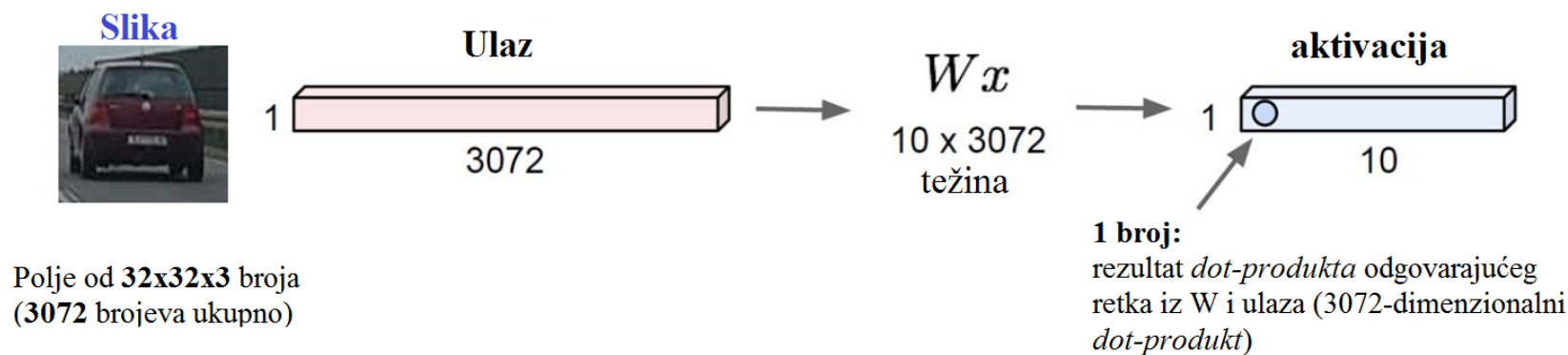
- Često ćete čuti da se neki sloj mreže zove POTPUNO POVEZANI SLOJ →??
- Primjer → obrada slike



- Ova ulazna slika se prikaže u obliku vektora od 3072 elementa/broja

Višeklasna klasifikacija

- Mreža s jednim izlaznim neuronom → binarna klasifikacija
- Ovo je primjer mreže s 10 izlaznih neurona → 10-klasna klasifikacija



- Sjetimo se →

$$\begin{aligned}x \in C_1 & \quad y = (1, 0, 0, \dots, 0, 0, 0) \\x \in C_2 & \quad y = (0, 1, 0, \dots, 0, 0, 0) \\& \quad \vdots \\x \in C_{K-1} & \quad y = (0, 0, 0, \dots, 0, 1, 0) \\x \in C_K & \quad y = (0, 0, 0, \dots, 0, 0, 1)\end{aligned}$$

Višeklasna klasifikacija

- Primjer: klasifikacija objekata detektiranih prednjom kamerom na vozilu



C_1 -auto



C_2 -kamion



C_3 -znak

...

...



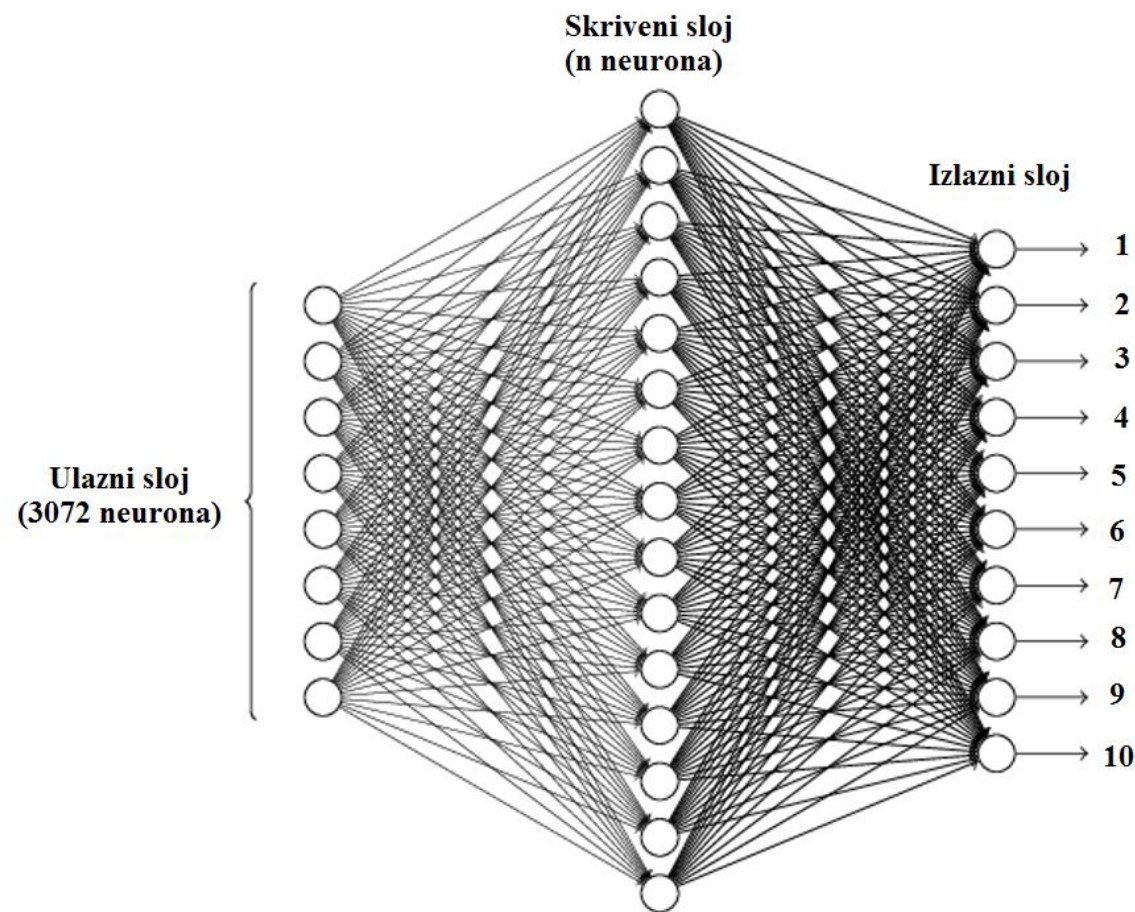
C_9 -pješak



C_{10} -semafor

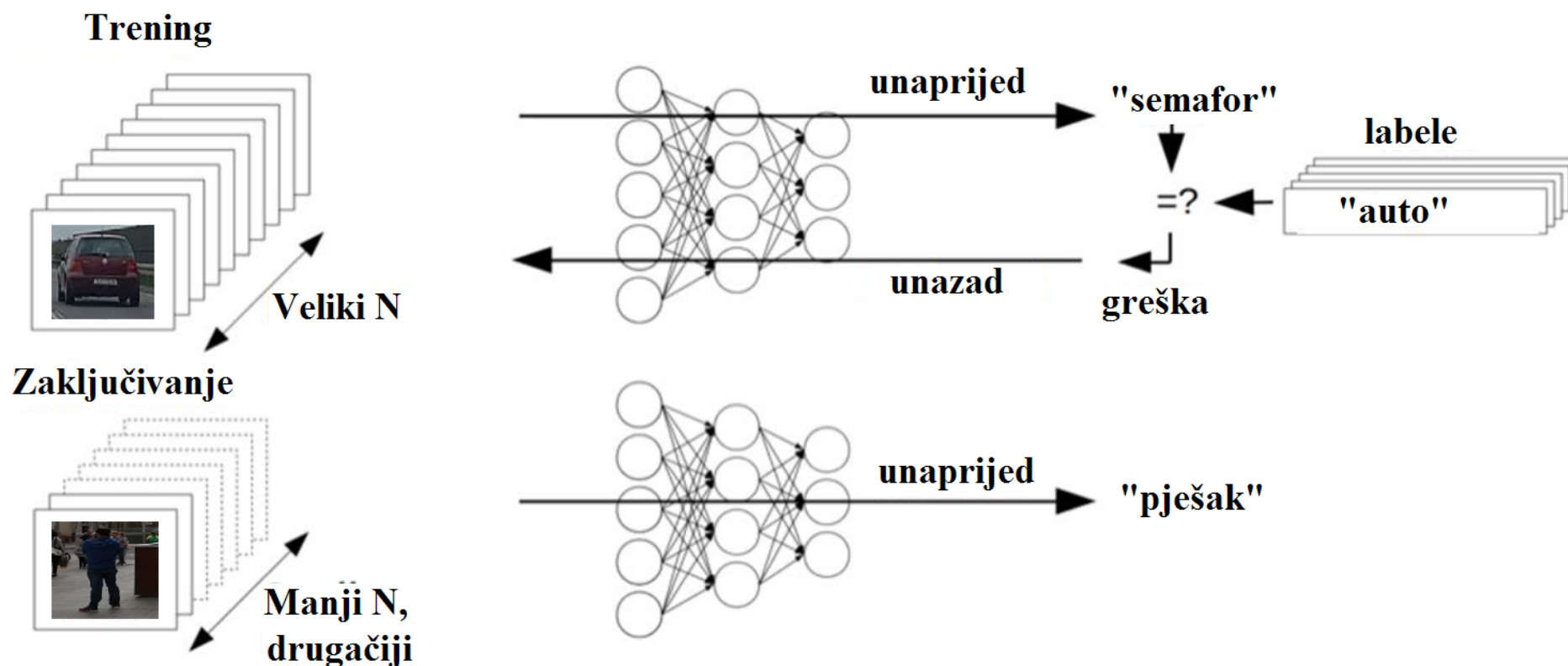
...

...



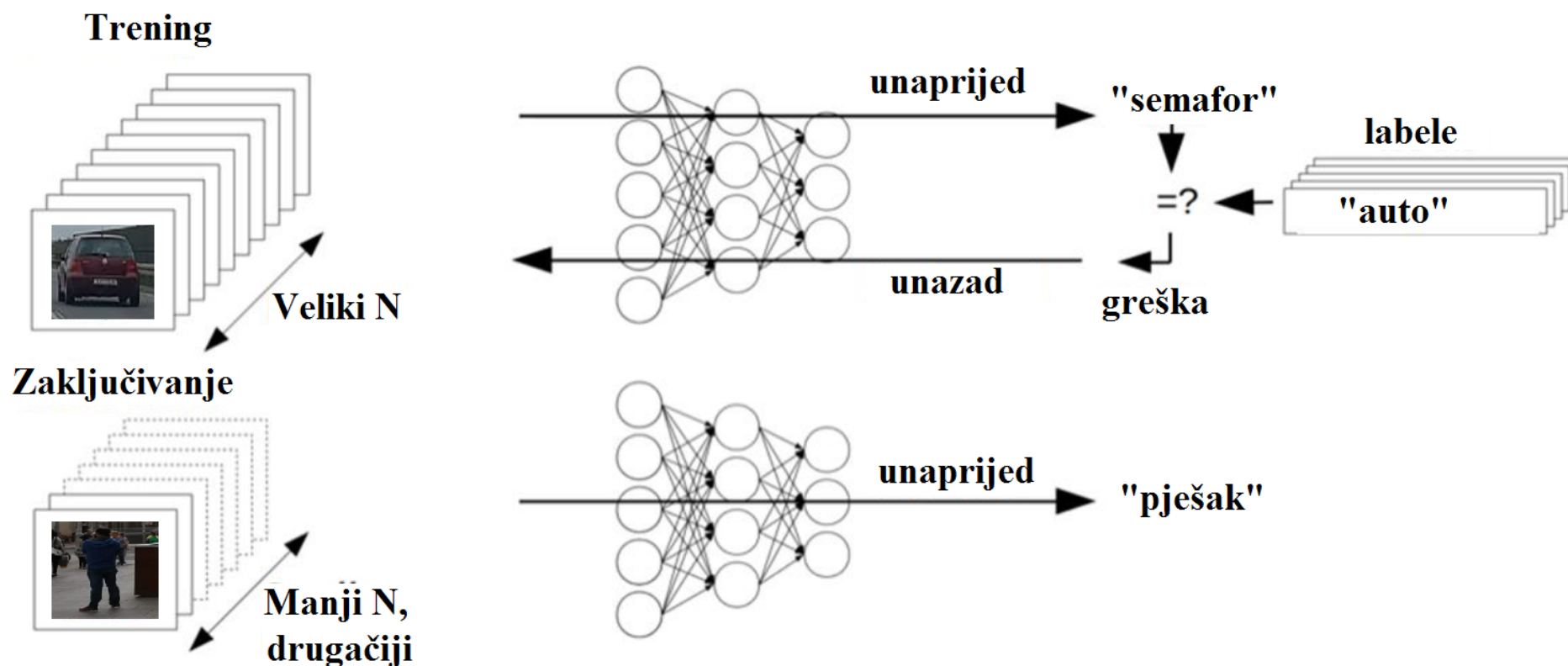
Treniranje i zaključivanje

- Svaka NN prolazi kroz dvije faze → treniranje i zaključivanje
- Često je potrebno dosta inženjerskog vremena da se odredi arhitektura NN (broj slojeva i neurona u pojedinom sloju) – a kad to odredimo, još ne znamo koeficijente mreže



Treniranje i zaključivanje

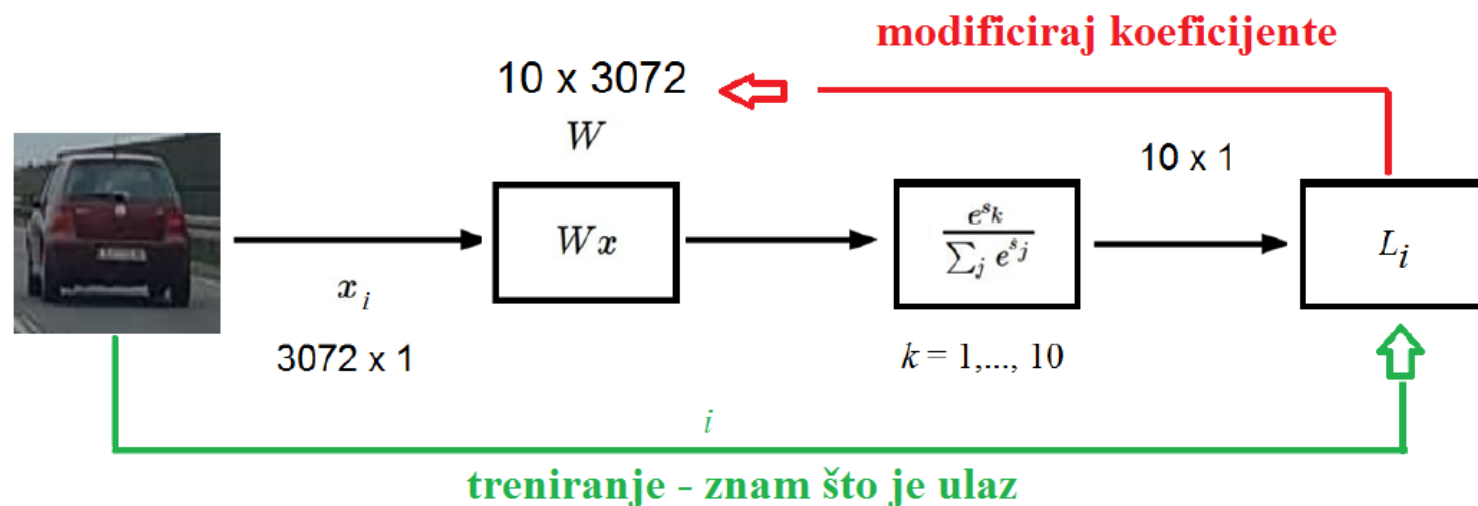
- Adaptacija koeficijenata u mreži unazad koristeći gradijentnu metodu (vidjet ćemo kasnije kako se i to radi) → zove se **BACK PROPAGATION METODA**
- Kad smo zadovoljni s % uspješnosti klasifikacije → verifikacija + testiranje



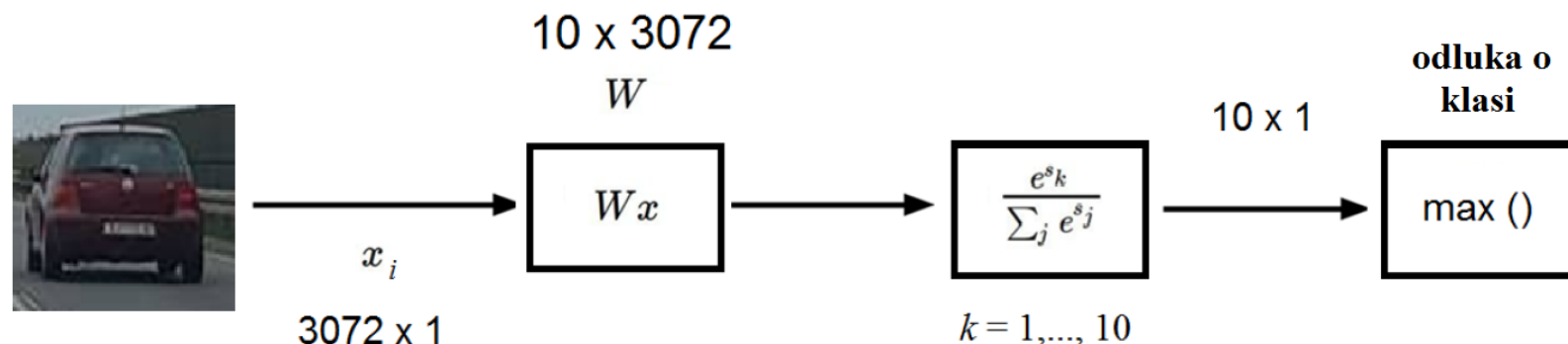
Podsjetnik: Linearni klasifikator i Logistička regresija

- Koja je razlika prikazanog na ovoj slici i prikazanog na slici s prethodnog slajda?

- Treniranje



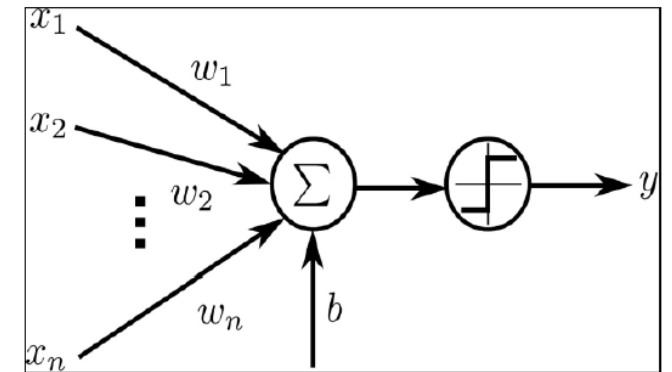
- Testiranje



P i t a n j a ?

Povijest primjene ANN...

- Prvi umjetni neuron (perceptron) → 1957.
- Ručno nagađanje koeficijenata
- Postojali su neuroni, mogle su se napraviti NN, čak i DNN, ali nije bilo jasno kako naučiti koeficijente ?!?!?
- 1980-ih → prvi ključan pomak ka široj primjeni DNN → pronađen način za adaptaciju (treniranje) koeficijenata u DNN → **BACK PROPAGATION METODA**
 - Ali još uvijek DNN nisu ostvarivale ni blizu svoj puni potencijal. Zašto ?
 - Nedostatak procesorske moći (GPU,...) – mjeseci, godine treniranja...
 - Nedostupnost velike količine podataka za treniranje
- 2006-2012. → drugi ključan pomak i *boom* DNN → riješena dva problema od ranije (CPU/GPU & Big Data)



Duboko učenje – općeniti komentari

- Pravi izazov za umjetnu inteligenciju, tj. DNN je rješavanje problema koji su ljudima jednostavni za riješiti, ali teški za opisati ih formalno (npr. jednadžbama) – problema koji su nama rješivi intuitivno
- Iako DL ima povijesne korijene desetljećima unazad, *boom* doživljava tek 2012. na klasifikaciji slika (1000 klasa)
 - Čovjek sliku opisuje bez problema (čak i malo dijete) – no možemo li to matematički opisati ?
 - Računalo sa jednom od najboljih mreža kaže ovo plavo napisano ispod slike!!!



Image Classification



a man sitting on a couch with a dog
a man sitting on a chair with a dog in his lap

Duboko učenje – općeniti komentari

- DL ubrzano postaje najbolja poznata tehnika AI
- DL je primjenjiv u mnogim aplikacijama
- Andrew Ng (2016): „*If a typical person can do a mental task with less than one second of thought, we can automate it using AI now or in the near future*”
 - „*AI is the new electricity – AI today is beginning to transform every major industry*”
- Kada se DNN istrenira, mi najvećim dijelom ne znamo što se unutra događa → Black Box

Moravec's Paradox: The “Easy” Problems are Hard



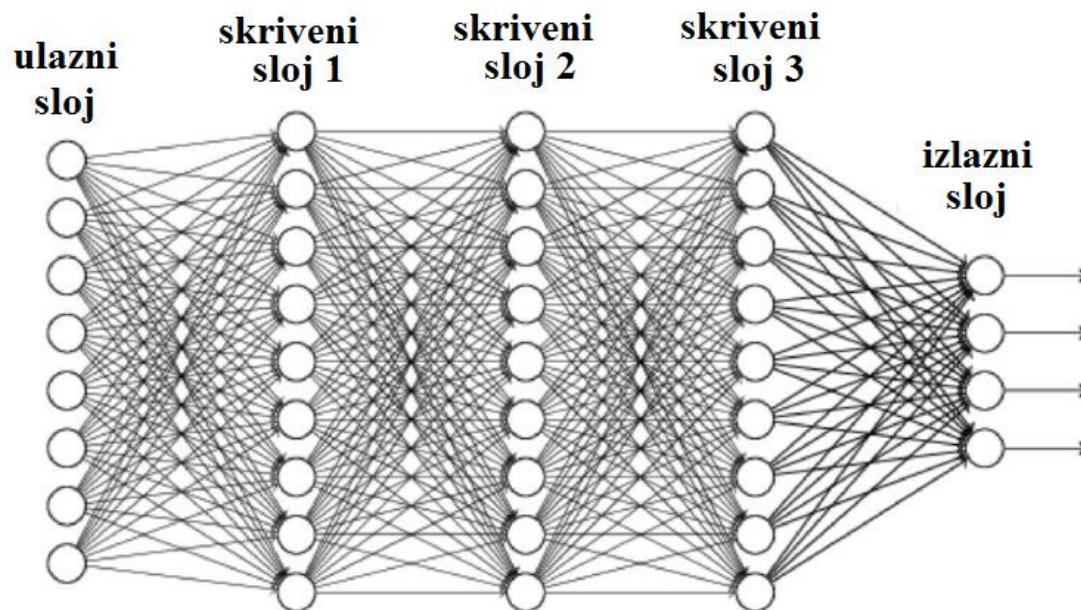
Soccer is harder than Chess



Is driving hard?

Duboke neuronske mreže – kompozicija funkcija

- Cilj DNN je aproksimirati funkciju $y = f(x, W)$
- Kroz proces učenja određuju se najbolji parametri W da se dobije najbolja aproksimacija $f^*(x, W)$
- Ta je funkcija zapravo kompozicija mnogo različitih funkcija (tj. slojeva)



$$y \sim f^*(x, \theta) = f^{(4)}(f^{(3)}(f^{(2)}(f^{(1)}(x))))$$

Univerzalnost neuronskih mreža

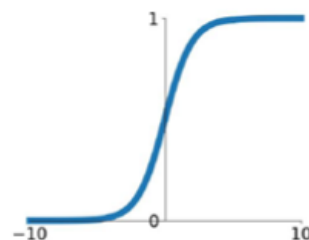
- Unaprijedne NN sa samo jednim slojem sposobne su dovoljno dobro implementirati bilo koju funkciju !!!
 - Ali moraju često sadržavati jako velik broj neurona u tom jedinom sloju
 - **UNIVERZALNO rješenje** za realizaciju bilo koje funkcije (linearne operacije + nelinearna aktivacijska funkcija)
 - Nelinearna aktivacijska funkcija smanjuje ograničenja nametnuta isključivo linearnim operacijama
- Umjesto jednog sloja s mnoštvom neurona → učinkovitije je koristiti više slojeva s manje neurona (dublje mreže) → manji je ukupni broj potrebnih neurona i operacija
- Pitanje – koji je smisao korištenja više neurona po sloju kad u sve njih ulaze isti ulazi?

Aktivacijske funkcije

- Na izlazu neurona nelinearna funkcija koja ima formu praga (oko nule)
- Mogu se koristiti razne aktivacijske funkcije → mnoga istraživanja provedena

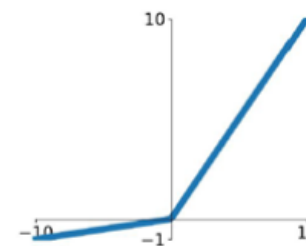
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



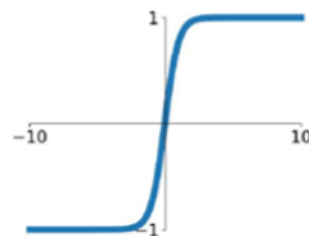
Leaky ReLU

$$\max(0.1x, x)$$



tanh

$$\tanh(x)$$

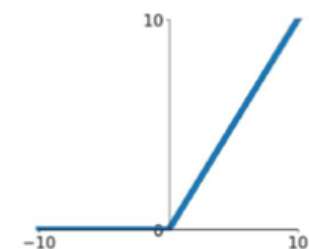


Maxout

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

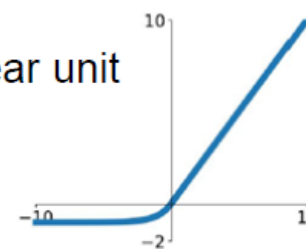
ReLU

$$\max(0, x)$$



ELU exponential linear unit

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$

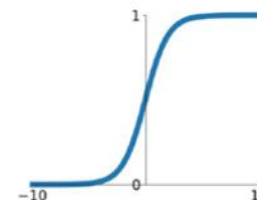


Aktivacijske funkcije

- Sigmoidna funkcija
 - Veliki negativni signal po apsolutnoj vrijednosti → izlaz 0
 - Pozitivne vrijednosti iznad praga → izlaz 1
 - Slična funkciji *sign*, ali je derivabilna
- ReLU (Rectified Linear Unit) funkcija
 - Za negativne ulazne vrijednosti → izlaz 0
 - Linearno raste za pozitivne vrijednosti
- Leaky ReLU funkcija
 - Modifikacija ReLU u negativnom dijelu ulaznih vrijednosti
- ELU (Exponential Linear Unit) funkcija
 - Linearna u „+” dijelu
 - Eksponencijalna u „-” dijelu

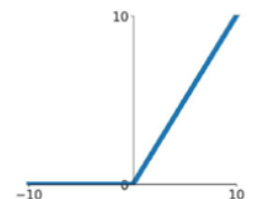
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



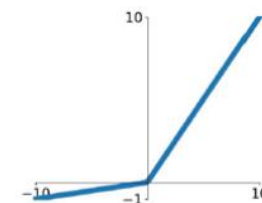
ReLU

$$\max(0, x)$$



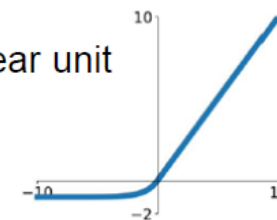
Leaky ReLU

$$\max(0.1x, x)$$



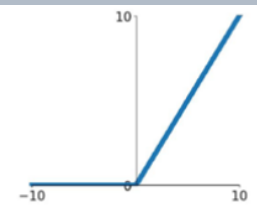
ELU exponential linear unit

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$

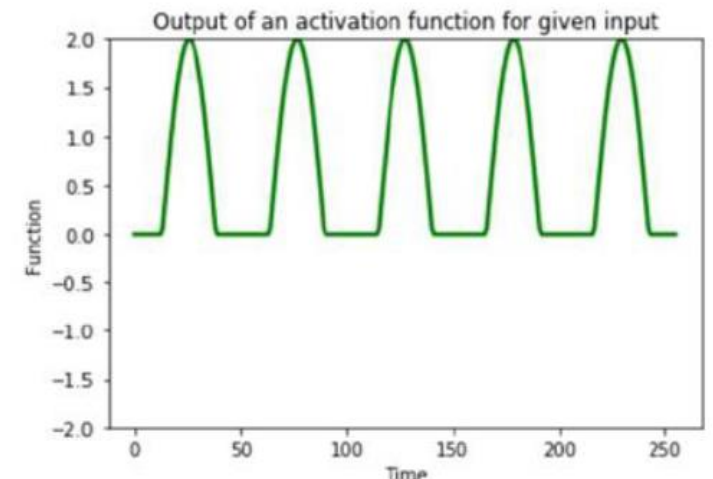
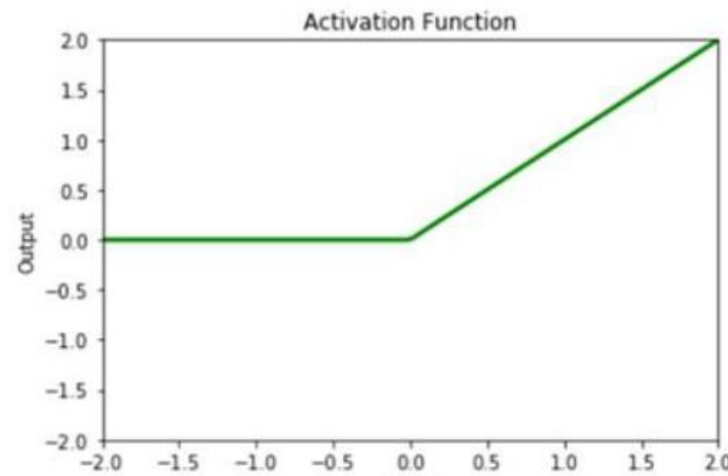
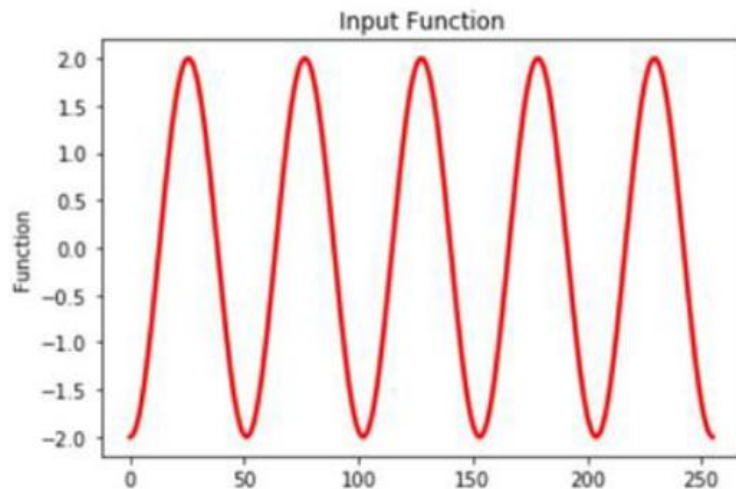


Rectified Linear Unit (ReLU)

$$\text{ReLU} \\ \max(0, x)$$



- Postala standard u DNN
- Rectified → „ispravlja” negativne vrijednosti na 0, a pozitivne propušta kakve jesu

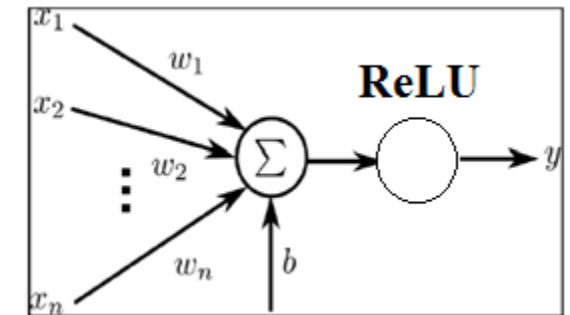
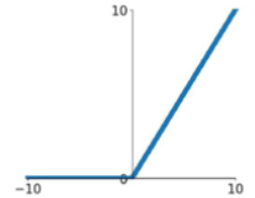


- ReLU → radi stvarno dobro, ali nema realnog objašnjenja zašto

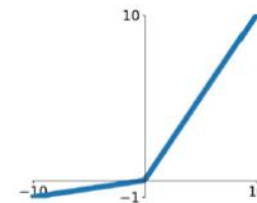
Rectified Linear Unit (ReLU) – problemi?

- Bitan izbor aktivacijske funkcije
- Empirijski pokazano → u početnoj fazi koeficijenti w_{ij} mogu biti izabrani na način da cijela mreža „zamre” (slučajno se biraju, mogu biti jako blizu 0) iako je trening uzorak relevantan
- Negativne vrijednosti i male pozitivne vrijednosti oko nule se propagiraju dalje kroz mrežu i mreža polako „zamire”
- Rješenja
 - Dodavanje *biasa*
 - Leaky ReLU ili ELU

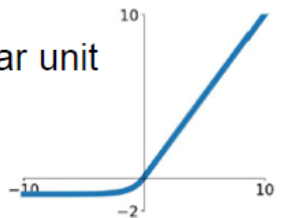
ReLU
 $\max(0, x)$



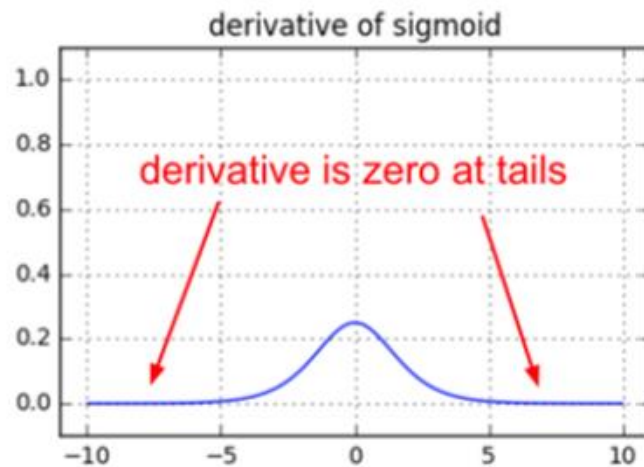
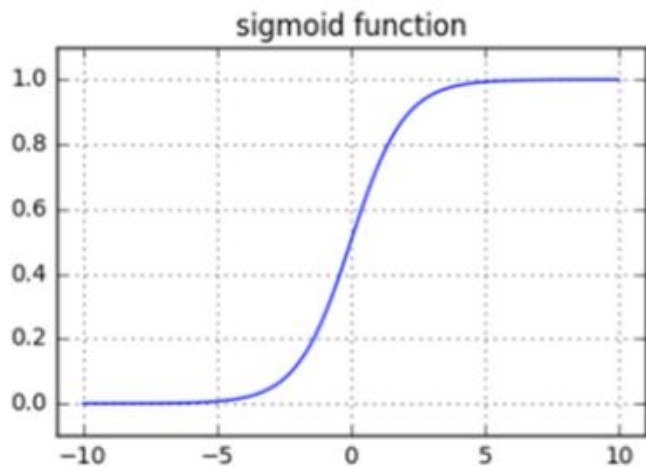
Leaky ReLU
 $\max(0.1x, x)$



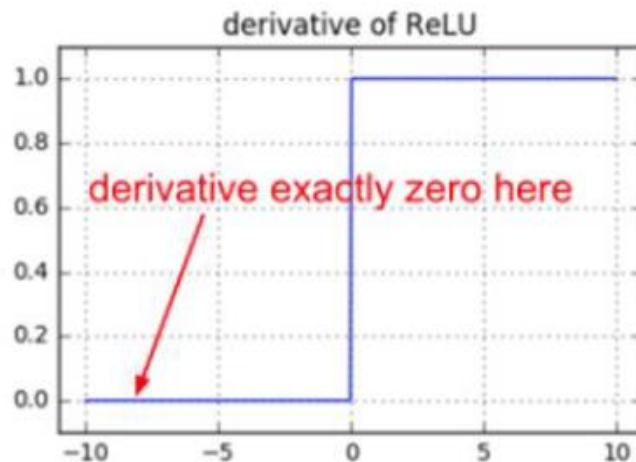
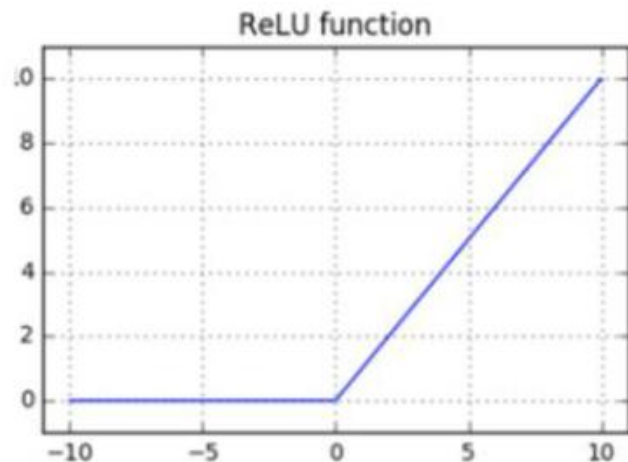
ELU exponential linear unit
$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



Aktivacijske funkcije i derivacije



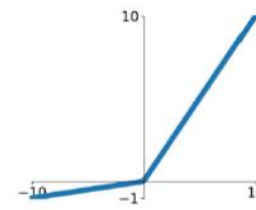
- Male derivacije vode do sporog procesa učenja



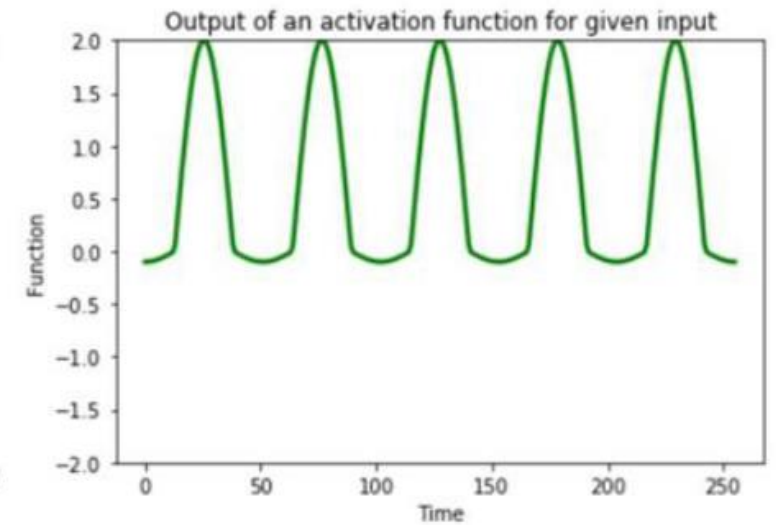
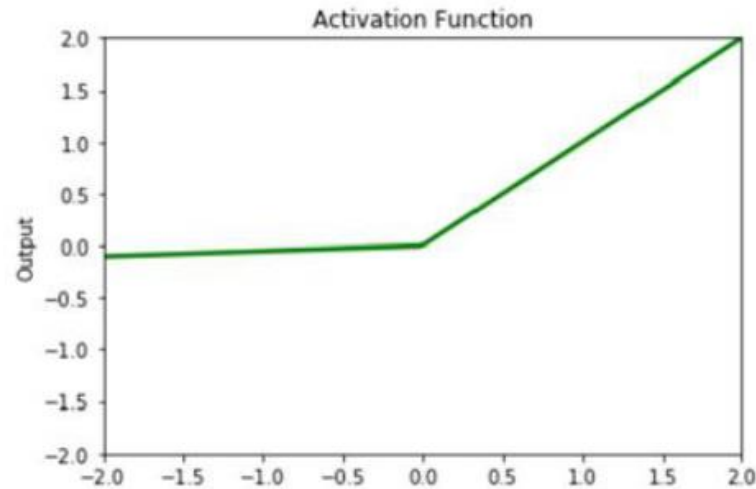
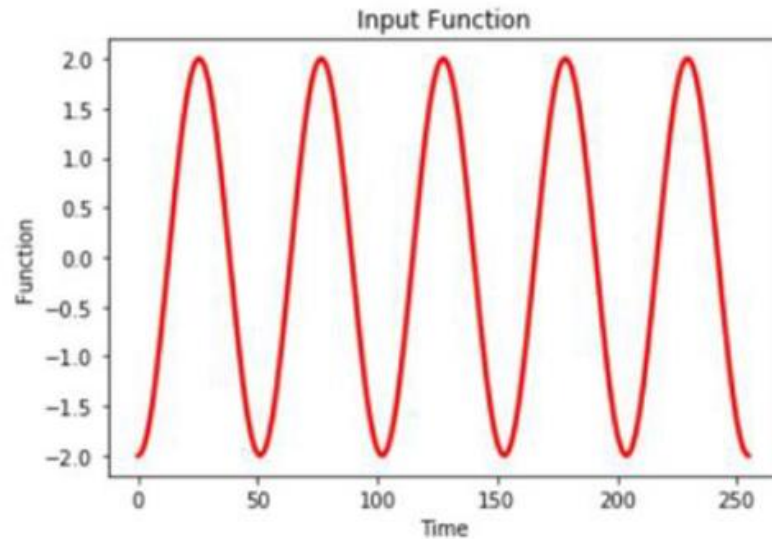
- Za krivo inicijalizirane početne vrijednosti mreža može „zamrijeti” na danom skupu za učenje

Leaky ReLU

Leaky ReLU
 $\max(0.1x, x)$



- Zbog ranije spomenutih problema → Leaky ReLU



- Na vježbama → primjena različitih aktivacijskih funkcija

P i t a n j a ?

Uč enje neuronske mreže

Podsjetnik... Iterativni proces učenja

METODA NAJBRŽEG SPUSTA

- Trening podaci $(x^{(i)}, y^{(i)})$

- Koeficijenti modela θ

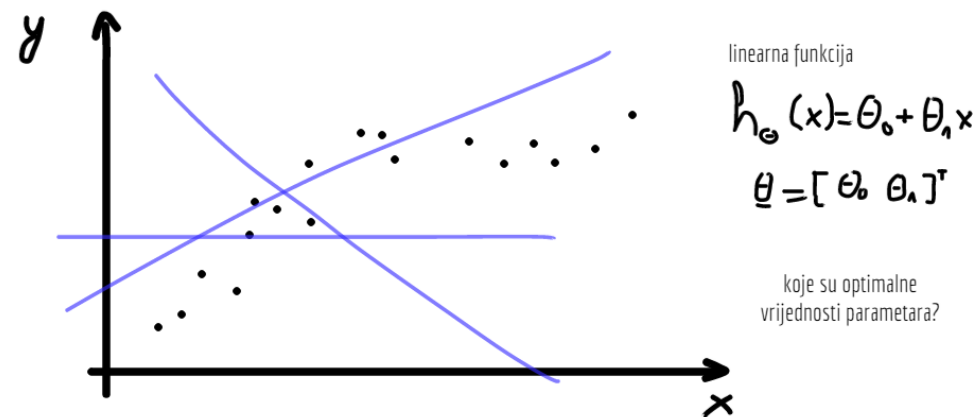
- Funkcija greške
(tzv. Cost function)

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

- Gradijent funkcije greške $\nabla_{\theta} J(\theta) = \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x^{(i)}$

- Ažuriranje koeficijenata

$$\theta(t+1) = \theta(t) - \alpha \nabla_{\theta} J(\theta(t))$$



Hipoteza: $h_{\theta}(x) = \theta_0 + \theta_1 x$

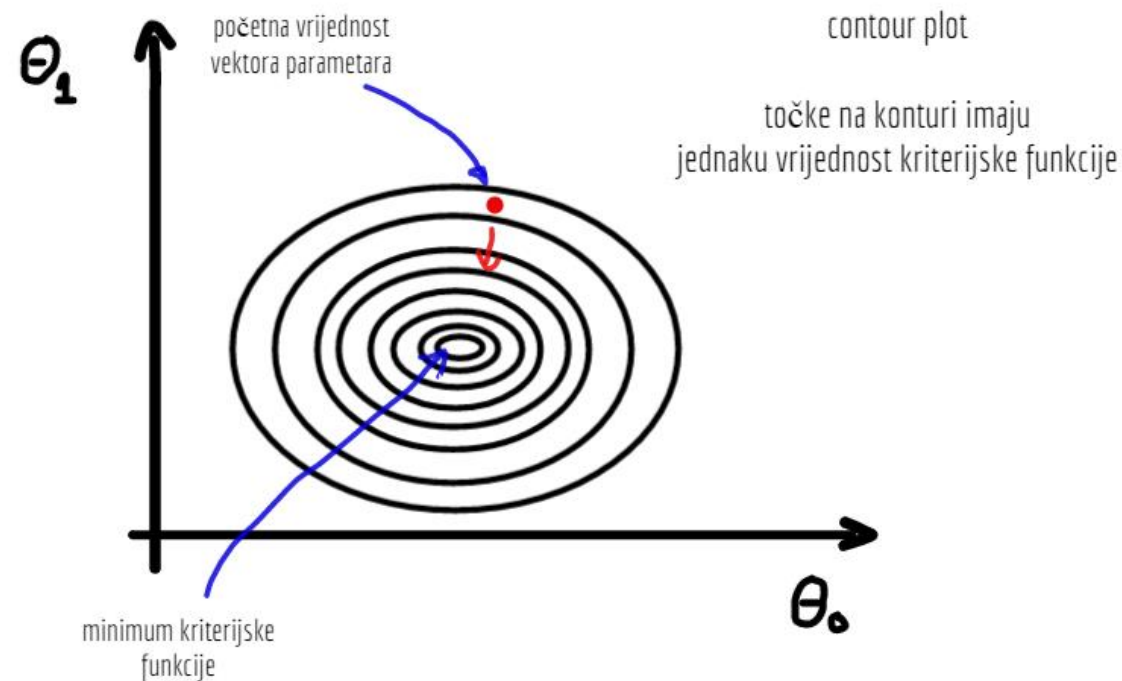
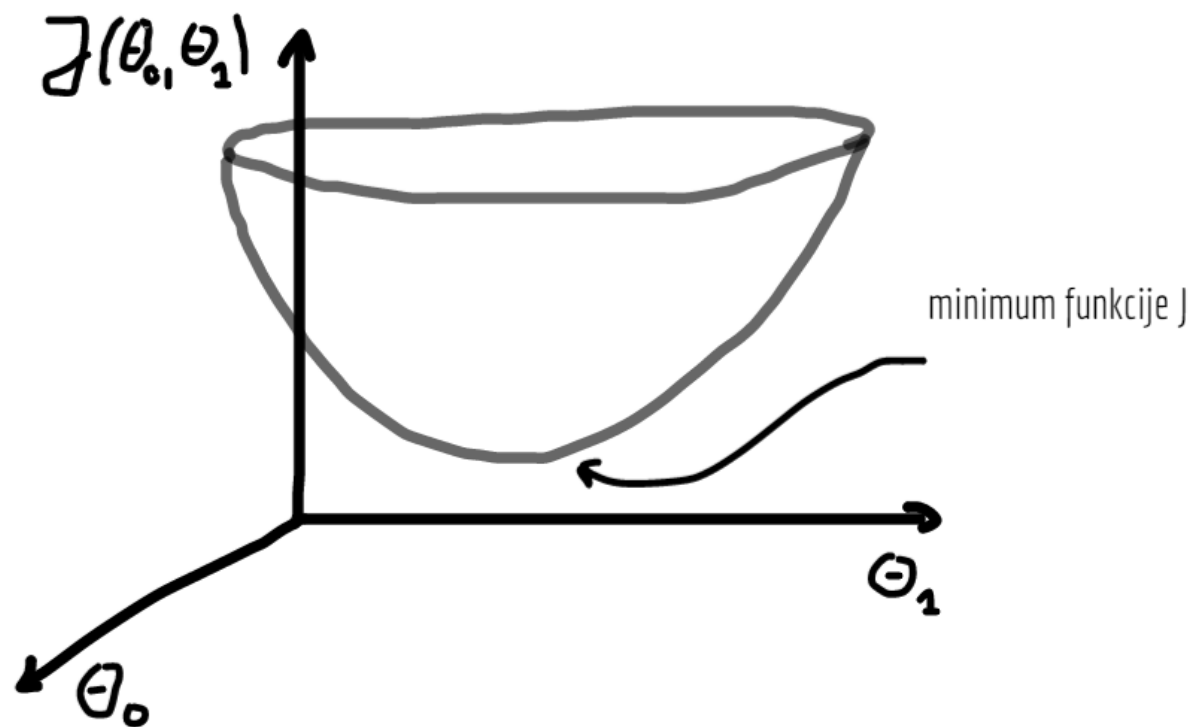
Parametri: θ_0, θ_1

Cost f-ja:

$$J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

Cilj: minimize $J(\theta_0, \theta_1)$

Podsjetnik... Iterativni proces učenja



Učenje neuronske mreže

- Učenje NN je složeniji problem od linearne funkcije
- Ulaz u mrežu je slika
 - N-dimenzionalni problem
 - Prolazi kroz sve neurone → jako velik broj koeficijenata w_{ij}
- Osim toga, NN predstavlja kompoziciju funkcija
 - Teže adaptirati nego običnu funkciju
- Pod pojmom učenje neuronske mreže podrazumijeva se
 - Strukturiranje neuronske mreže (**odabir broja slojeva, odabir broja neurona u pojedinom sloju, odabir tipa aktivacijske funkcije neurona**)
 - Procjena nepoznatih parametara neuronske mreže, tj. težina mreže → **provodi se minimizacijom odgovarajuće kriterijske funkcije**

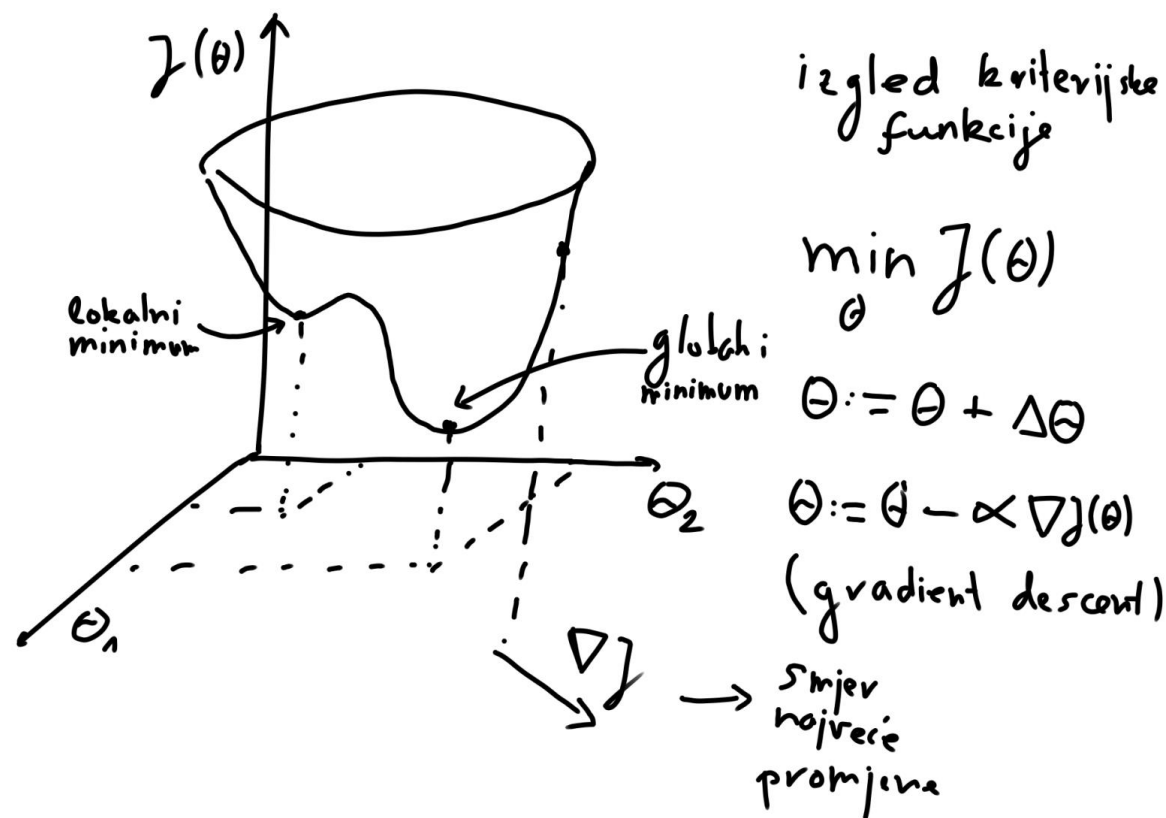
Slika



Polje od **32x32x3** broja
(**3072** brojeva ukupno)

Učenje neuronske mreže

- Pronaći vektor/matricu parametara w_{ij} mreže koji minimizira kriterijsku funkciju
- Kriterijska funkcija je izrazito nelinearna funkcija s obzirom na parametre (težine) mreže
- Mnogo je lokalnih minimuma
- Nije moguće pronaći analitičko rješenje optimizacijskog problema
- Koriste se iterativni numerički postupci
- **Metoda gradijentnog spusta → promjena parametara u smjeru negativnog gradijenta kriterijske funkcije**

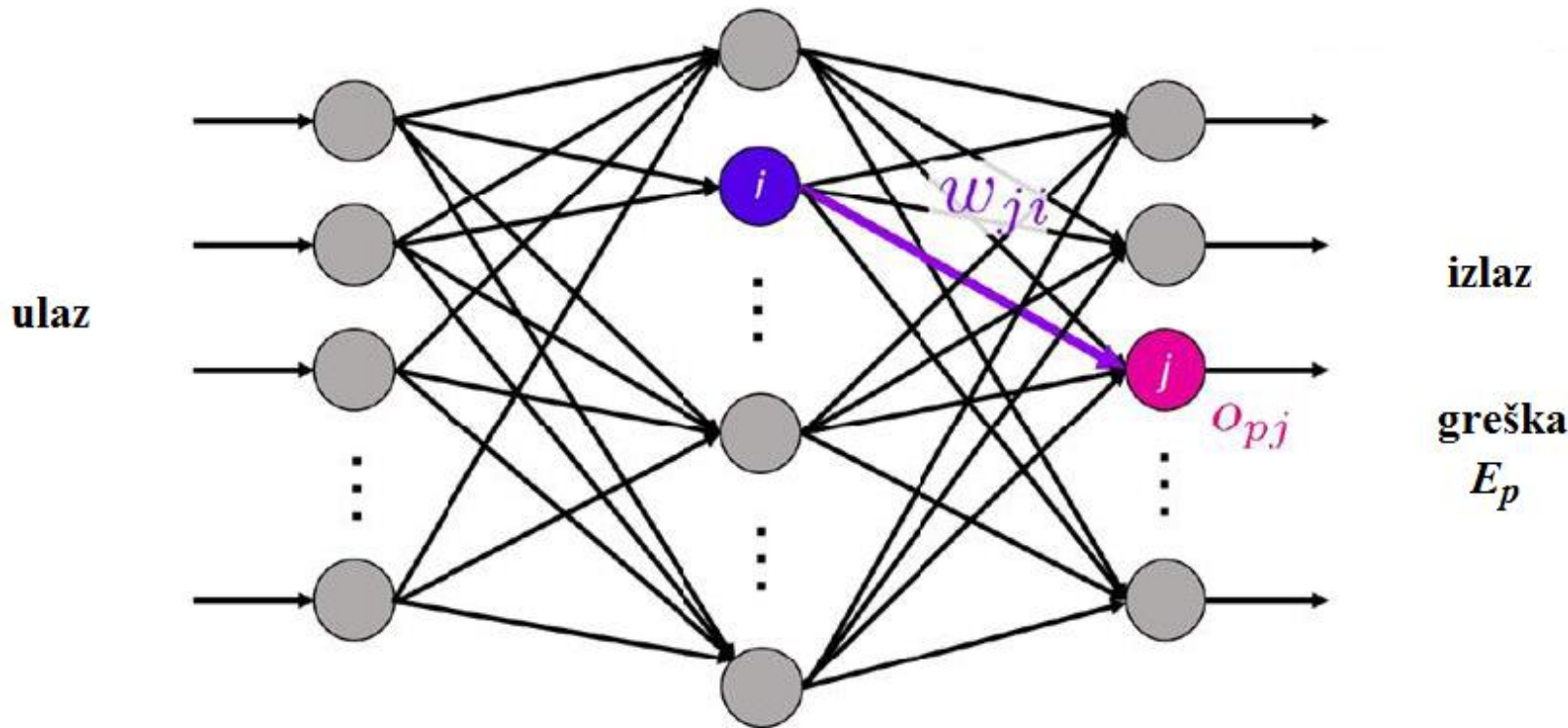


BACK-PROPAGATION algoritam

- Algoritam je originalno predstavljen u 70-im godinama, ali popularnost dobiva tek s radom Davida Rumelharta iz 1986.
- Ovaj algoritam je “motor” za postupak učenja neuronske mreže
- Omogućuje brzo izračunavanje gradijenta kriterijske funkcije na temelju kojeg se pre-podešavaju parametri mreže
- “*the backward propagation of errors*” – izračunava se pogreška mreže za određeni podatak te se distribuira unazad kroz slojeve mreže
- Dvije pretpostavke vezane za kriterijsku funkciju:
 - Može se zapisati kao srednja vrijednost pogreške
 - Može se zapisati kao funkcija izlaza neuronske mreže
- Back-propagation algoritam sastoji se od dva glavna koraka:
 - Unaprijedna propagacija
 - Ažuriranje parametara

BACK-PROPAGATION algoritam

- Često se krivo shvaća da je to algoritam specifičan za višeslojne NN, ali u principu on može izračunati derivacije bilo koje funkcije



$$\frac{\partial E_p}{\partial w_{ji}} = \frac{\partial E_p}{\partial o_{pj}} \frac{\partial o_{pj}}{\partial w_{ji}}$$

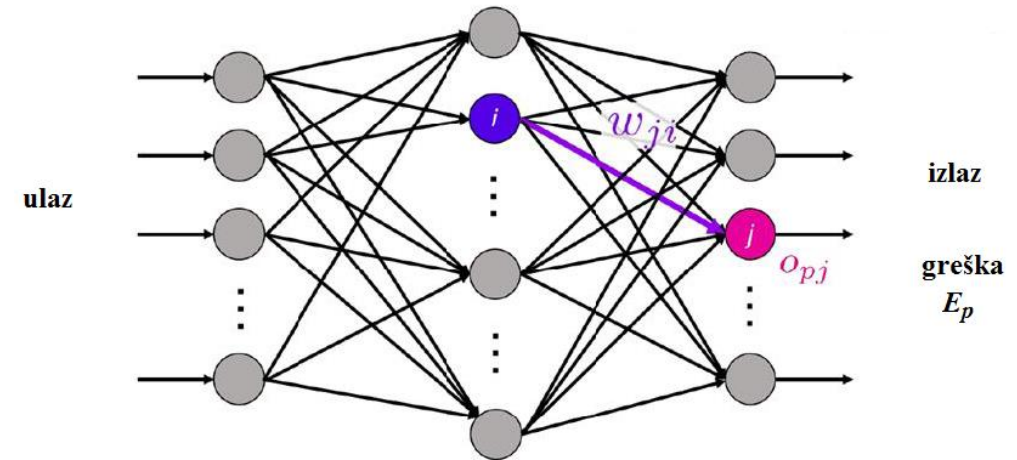
Rumelhart et. al. 1986

BACK-PROPAGATION algoritam

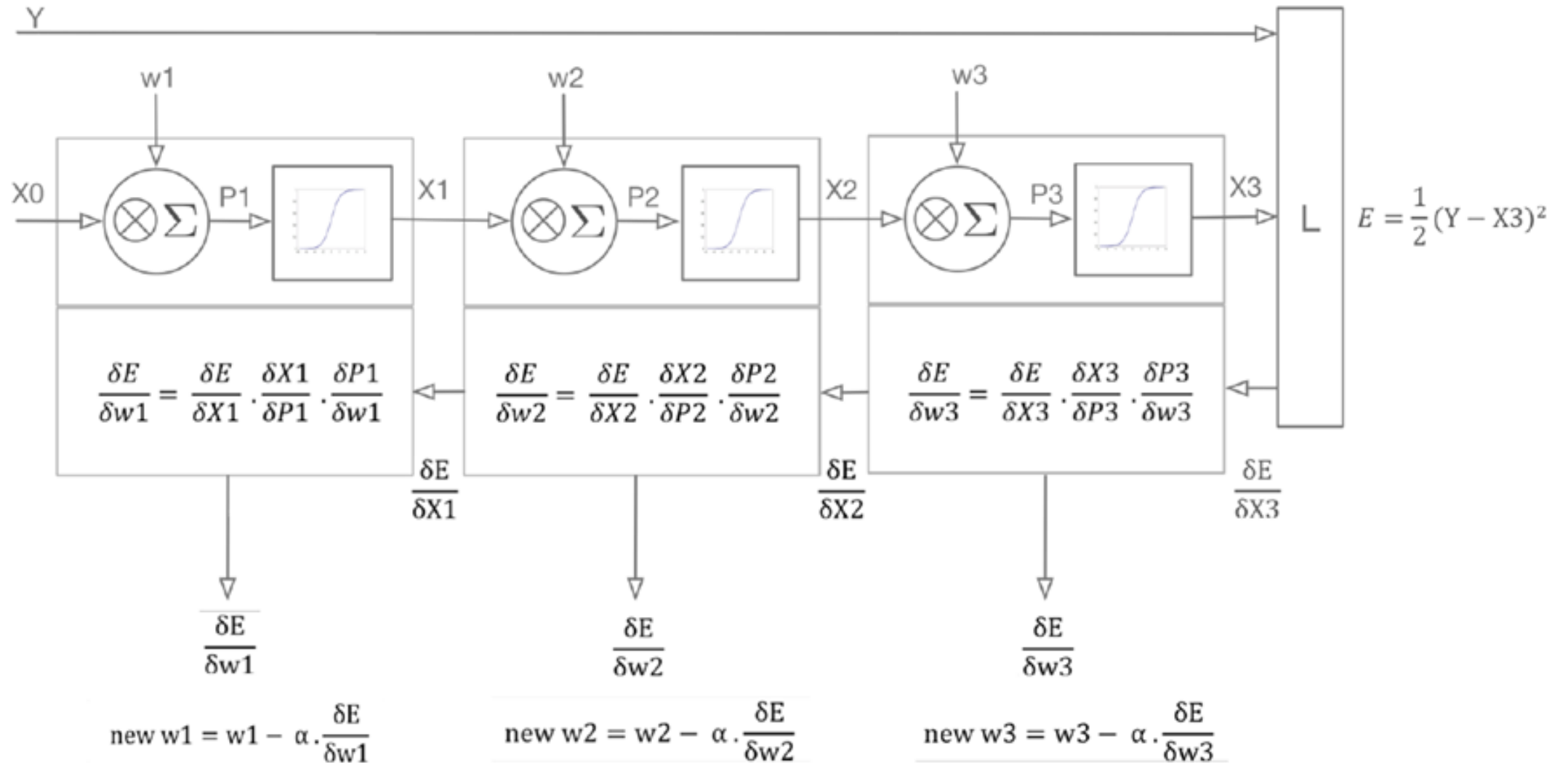
- Što nas zanima → kako promijeniti parametre w_{ji} da se greška (iznos *cost* funkcije) smanji ?
- Činjenica
 - Funkcija greške „ne vidi” direktno w_{ji}
 - Vidi direktno izlaz o_{pj}
- Pravilo ulančavanja derivacije

$$\frac{\partial E_p}{\partial w_{ji}} = \frac{\partial E_p}{\partial o_{pj}} \frac{\partial o_{pj}}{\partial w_{ji}}$$

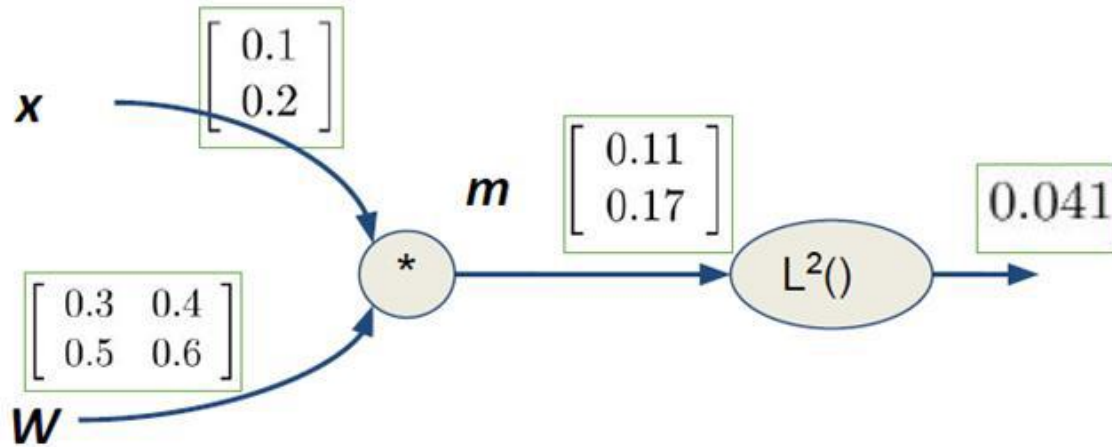
- Ovo je samo prvi korak ... → ide se unazad i koristi se isto pravilo deriviranja
- **Ovo je jedan od ključnih koraka koji je pomjerio ANN s fiksnih (ručno određenih) parametara na one koji se mogu iterativno mijenjati**



BACK-PROPAGATION algoritam



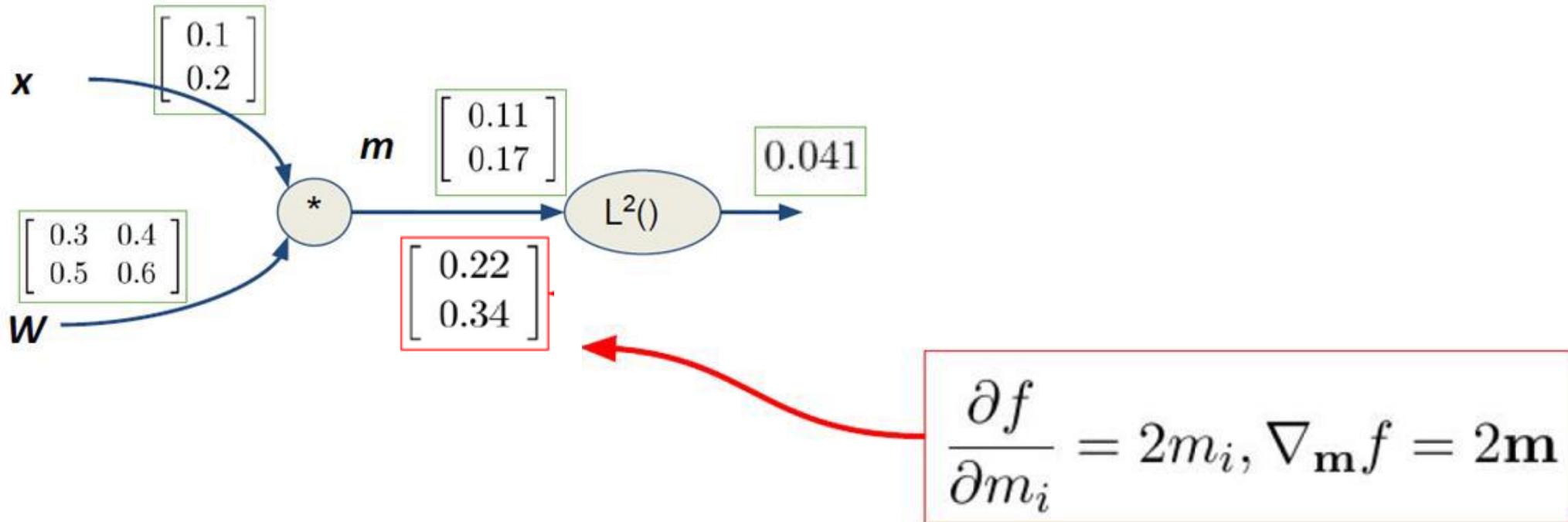
BACK-PROPAGATION algoritam



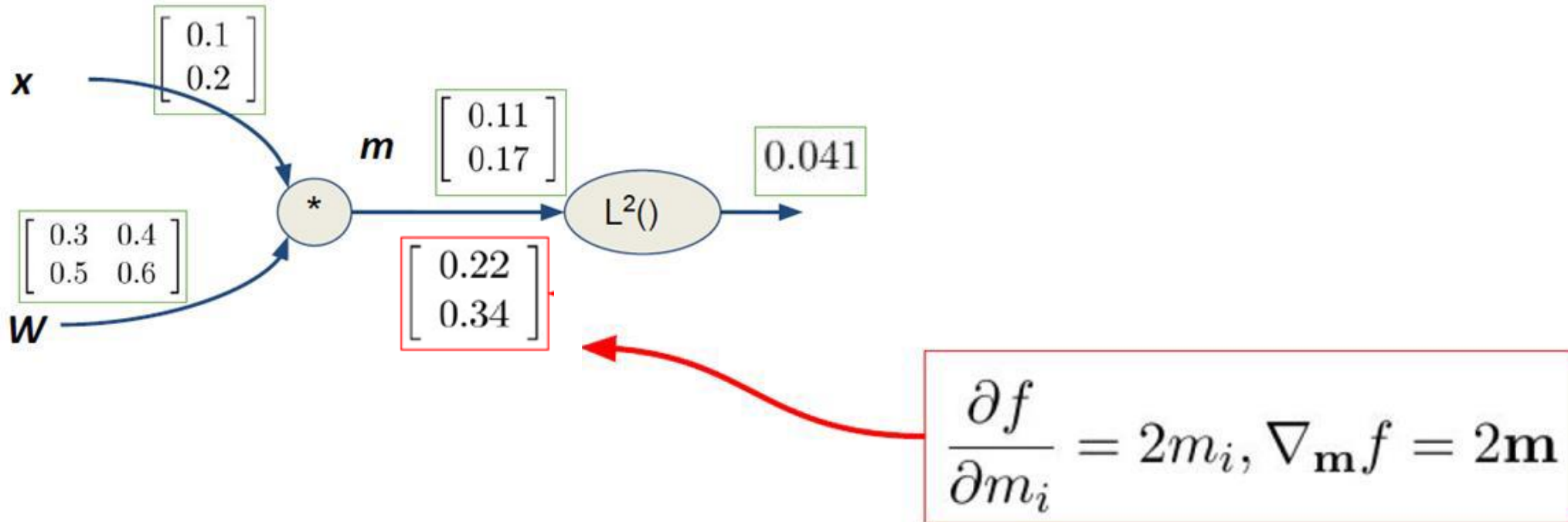
- Neka imamo na ulazu 2D vektor
- Neka smo odabrali sasvim slučajno matricu početnih parametara W
- I neka je naš izlaz definiran kao $m=Wx \rightarrow m$ je također 2D vektor
- Potpuno ilustrativno $\rightarrow$ neka u ovom slučaju koristimo **kriterijusku funkciju** $L^2()$ tj. sumu kvadrata svih komponenti vektora m kao mjeru greške $\rightarrow$ ona treba inicirati promjenu koeficijenata W

- $L^2()$ ne vidi direktno W , ali vidi m (to je njezin ulaz) $\rightarrow L^2()=m^2$
- Vrlo lako se izračuna derivacija te funkcije greške po m

BACK-PROPAGATION algoritam



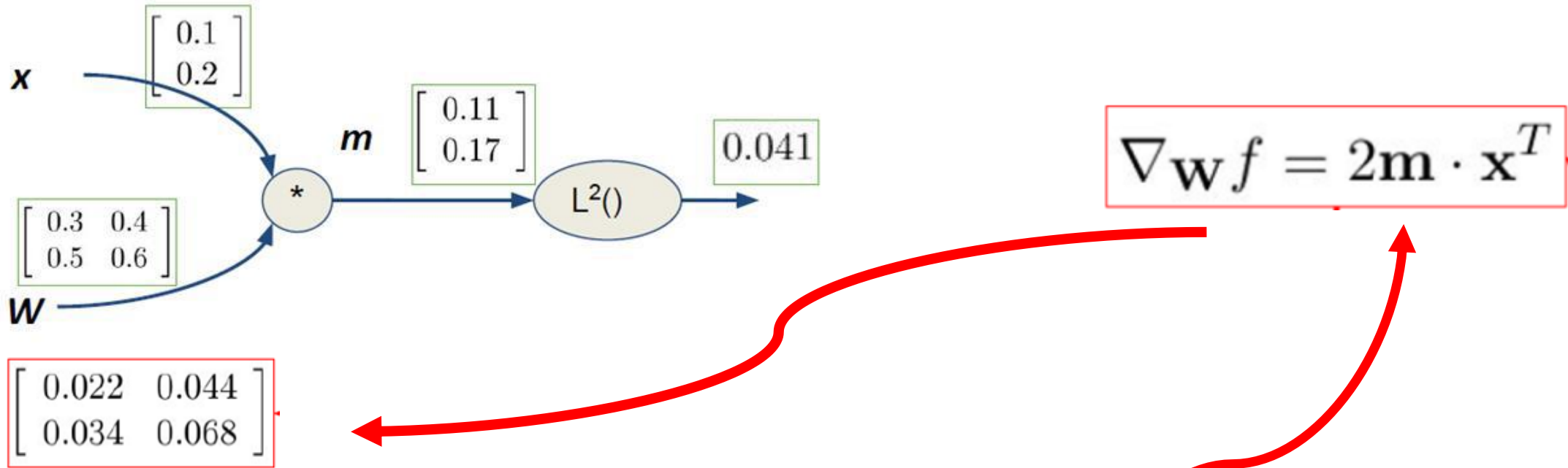
BACK-PROPAGATION algoritam



$$\frac{\partial m_k}{\partial W_{i,j}} = \mathbf{1}_{k=i} x_j$$

$$\frac{\partial f}{\partial W_{i,j}} = \sum_k \frac{\partial f}{\partial m_k} \frac{\partial m_k}{\partial W_{i,j}} = \sum_k (2m_k) (\mathbf{1}_{k=i} x_j) = 2m_i x_j$$

BACK-PROPAGATION algoritam



$$\frac{\partial m_k}{\partial W_{i,j}} = \mathbf{1}_{k=i} x_j$$

$$\frac{\partial f}{\partial W_{i,j}} = \sum_k \frac{\partial f}{\partial m_k} \frac{\partial m_k}{\partial W_{i,j}} = \sum_k (2m_k) (\mathbf{1}_{k=i} x_j) = 2m_i x_j$$

Modifikacije gradijentne metode najbržeg spusta

- U praksi, izračunavanje gradijenta kriterijske funkcije na temelju predstavljene *batch* metode može potrajati zbog velikog broja mjernih uzoraka
- Kako bi se ubrzao proces učenja može se koristiti stohastička metoda najbržeg spusta (engl. *stochastic gradient descent*)
- Ideja je aproksimirati gradijent računanjem gradijenta kriterijske funkcije na jednom podatku ili na malom uzorku mjernih podataka (engl. *mini-batch stochastic gradient descent*)
- Pri tome je potrebno nasumično odabirati mjerne podatke sve dok se ne iskoriste svi mjerni podaci kada kažemo da je završila jedna epoha učenja

Neki praktični savjeti

- Inicijalizacija parametara mreže
 - Metode optimizacije zahtijevaju početnu vrijednost vektora parametara
 - Obično se težine generiraju nasumično u nekom malom intervalu oko 0
- Metoda gradijentnog spusta osjetljiva je na prisutnost lokanih minimuma
 - Ovo obično nije toliko značajan problem jer mreža često dobro radi i sa parametrima nekog lokalnog minimuma
 - Moguće je nekoliko puta pokrenuti učenje mreže s različitim početnim vrijednostima parametara
- Kako spriječiti *overfitting*?
 - Koristiti veći broj neurona u mreži
 - Koristiti validacijski set
 - Prikupiti još podataka za učenje
- Dodavanje slojeva → oprez → može dovesti do premale promjene koeficijenata, a uz to ona i duže traje
- Obično 10-15 slojeva dovoljno (2016 Microsoft → oko 150 slojeva, a koeficijenti se ipak mijenjaju!)

P i t a n j a ?